Parallel-machine scheduling with weighted earliness-tardiness and optional job rejection

Ren-Xia Chen[a], Shi-Sheng Li[a,*], Cheng-Wen Jiao[a]

[a]School of Mathematics and Information Science, Zhongyuan University of Technology, Zhengzhou 450007, People's Republic of China

Abstract: We address the problem of scheduling n jobs on m identical parallel machines with an unrestricted common due date, while permitting job rejection. The objective is to minimize the integrated cost, comprising the total weighted earliness and tardiness of accepted jobs along with the total rejection penalty. We show that when each job's weight is equal to its processing time, the single-machine case is binary $\mathcal{NP}$-hard, whereas the parallel-machine variant is strongly $\mathcal{NP}$-hard for arbitrary m even without rejection. For fixed m, we first design two pseudo-polynomial dynamic programming algorithms, and then convert them into two fully polynomial-time approximation schemes. Their time complexities are $O(\frac{n^{2m+1}}{\epsilon^{2m}}(\log P_{\text{sum}})^{2m})$ and $O(\frac{n^{2m+1}}{\epsilon^{2m}}(\log W_{\text{sum}})^{2m})$, where P_{sum} and W_{sum} denote the total processing time and the total weight of all jobs, respectively. Moreover, we propose $O(n^2)$-time algorithms for two special cases, one where all jobs share identical processing times and another where all jobs have equal weights.

Keywords: Scheduling; Parallel-machine; Earliness and tardiness; Rejection

1 Introduction

In modern manufacturing, just-in-time (JIT) production systems face the critical challenge of balancing operational efficiency with customer satisfaction. A fundamental aspect of this balance involves addressing earliness and tardiness (ET) costs, which occur when jobs are completed either before or after their due dates (Rolim and Nagano, 2020). Unlike traditional scheduling methods that focus on minimizing maximum lateness or total (weighted) comple-

*Corresponding author. E-mail address: liss@zut.edu.cn (S.S. Li)

tion times, ET optimization presents a JIT paradox, as both early and late job completions incur significant penalties. Earliness can lead to unnecessary expenses, including spoilage of perishable materials, inefficient labor utilization, and capital being tied up in excess inventory. Conversely, tardiness can result in customer dissatisfaction, contractual penalties, and damage to firm's reputation (Hall and Posner, 1991). For instance, an automotive supplier that completes engine assemblies too early may incur storage costs and quality risks, while late deliveries can disrupt assembly lines and lead to financial penalties. Similarly, a clothing factory overwhelmed by seasonal orders faces two interrelated challenges: retaining excess inventory consumes working capital and increases the risk of material degradation, while delayed deliveries threaten long-term client trust.

Due to its significant theoretical and practical implications, ET scheduling has attracted considerable interest from both scheduling theory researchers and industry practitioners. An important aspect of ET scheduling concerns scenarios where all jobs share a common due date (CON). Hall and Posner (1991) first showed that minimizing the total weighted earliness and tardiness (TWET) with an unrestricted CON on a single machine is binary $\mathcal{NP}$-hard, and then presented a pseudo-polynomial dynamic programming (DP) algorithm. Hall et al. (1991) and Hoogeveen and van de Velde (1991) demonstrated independently that the problem of minimizing the total earliness and tardiness (TET) with a restricted CON on a single machine is binary $\mathcal{NP}$-hard, and further provided pseudo-polynomial algorithms for the TET and TWET minimization problems, respectively. Li and Cheng (1994) studied the problem of minimizing the maximum weighted ET with an unrestricted CON on a set of m identical parallel machines. They showed that this problem is binary $\mathcal{NP}$-hard even for $m = 1$, and it is strongly $\mathcal{NP}$-hard for arbitrary m. Since then, a broad range of extensions and variations on ET scheduling have been investigated in the literature. For more comprehensive insights into this area, we refer the readers to the surveys by Kramer and Subramanian (2019) and Rolim and Nagano (2020). Recent research on ET scheduling with the CON model has explored various objective functions, solution methods, and machine settings. For instance, Kellerer et al. (2020) developed a fully polynomial-time approximation scheme (FPTAS) for the minimization of TWET with an unrestricted CON on a single machine. Li and Chen (2020) considered a single-machine scheduling with CON assignment to minimize generalized weighted ET penalties, which include variable costs dependent on job-specific earliness or tardiness and fixed costs incurred for each early or tardy job. Arik et al. (2022) proposed several heuristics and local search algorithms to minimize the TET for unrelated parallel machines with a restricted CON. Nasrollahi et al. (2022) considered a CON scheduling problem with two agents in a two-machine flow shop environment. Arik (2023) developed a heuristic algorithm to tackle the single-machine scheduling problem where the ET weights are job-dependent and the CON is assignable. Atsmony and Mosheiov (2024a)

studied a CON assignment scheduling problem with fixed-plus-linear ET costs on a single machine.

On the other hand, contemporary large-scale production systems often require decision-makers to balance inventory costs and service quality by rejecting certain orders or outsourcing some tasks to external partners. This process is known as order acceptance and scheduling (OAS) or machine scheduling with rejection (MSR). A practical example is found in the printing industry, where companies must manage diverse products with tight due dates and limited capacity, making coordinated job rejection and scheduling essential (Oğuz et al., 2010). Similarly, logistics systems that require specialized services within strict time windows compel providers to choose jobs based on profitability and the availability of time slots (Cesaret et al., 2012). The OAS/MSR framework significantly enhances scheduling flexibility for decision-makers by allowing job selection or order acceptance, in contrast to traditional models that require processing every job without exception. Slotnick (2011) examined trade-offs between revenue and cost in selective order processing, providing a literature review and classification framework for research in OAS. Shabtay et al. (2013) presented a comprehensive survey of offline MSR, highlighting the need in make-to-order systems to balance production costs and job rejection. The OAS/MSR model is applicable across various domains, as evidenced by the range of models in the literature. Recent studies on this aspect have explored several themes, including job shop scheduling (Safarzadeh and Kianfar, 2019; Chen and Li, 2024), flow shop scheduling (Kim and Lee, 2022; Shabtay and Gerstl, 2024), due date assignment scheduling (Mor and Shapira, 2022; Atsmony and Mosheiov, 2024b; Mosheiov and Sarig, 2024), prize-collecting scheduling (Hou et al., 2024; Hu et al., 2024), and scheduling with release dates (Kacem and Kellerer, 2024; Geng and Lu, 2025).

While ET scheduling and MSR are well-established research domains individually, their integration remains relatively unexplored. Atsmony and Mosheiov (2024b) addressed a CON assignment scheduling problem with job rejection on a single machine, aiming to minimize the maximum earliness and tardiness of accepted jobs subject to a given budget on total penalties of rejected jobs. They showed that this problem is $\mathcal{NP}$-hard and provided a pseudo-polynomial algorithm. Mosheiov and Sarig (2024) examined a CON assignment problem with equal processing times and optional job rejection on two uniform parallel machines. The objective is to minimize a combined cost that includes ET cost, due date assignment cost, and rejection penalties. They provided a polynomial algorithm to solve it.

In this paper, we investigate a generalized ET scheduling problem with four key features: (i) an unrestricted CON for all jobs; (ii) job-specific ET weights; (iii) optional job rejection; and (iv) m identical parallel machines. The objective is to minimize the sum of TWET of accepted jobs and total penalty of rejected ones. For the single-machine case without

rejection, Hall and Posner (1991) gave a linear-time algorithm under proportional weights. In contrast, we prove that adding rejection makes this case binary $\mathcal{NP}$-hard. We further prove that the parallel-machine variant is strongly $\mathcal{NP}$-hard for arbitrary m, even without rejection. For fixed m, we propose pseudo-polynomial DP algorithms and FPTASs. Unlike the single-machine scenario, these algorithms need to track early and tardy variables on each machine separately, which causes the state space to grow multiplicatively with m.

The rest is structured as follows. Section 2 gives a detailed description of the problem under study. Section 3 analyzes structural properties that characterize optimal solutions. Section 4 demonstrates two $\mathcal{NP}$-hardness results. Section 5 develops pseudo-polynomial DP algorithms and FPTASs for fixed number of identical machines. Section 6 designs polynomial algorithms to solve two special cases. Section 7 concludes the paper and points out directions for future research.

2 Problem description

Consider a set $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ of n independent jobs, all of which arrive simultaneously at time zero. The processing system comprises a set $\mathcal{M} = \{M_1, M_2, \ldots, M_m\}$ of m identical parallel machines. Each job J_j $(j = 1, 2, \ldots, n)$ is characterized by a triple (p_j, w_j, r_j), where $p_j > 0$ denotes the processing time, $w_j > 0$ represents the weight, and $r_j > 0$ indicates the rejection cost. All parameters p_j, w_j, and r_j are assumed to be integers. For each job J_j, the decision-maker faces two options: either processes the job nonpreemptively on one of the m machines in $\mathcal{M}$, or rejects it, incurring a penalty cost of r_j. For any given subset $\mathcal{X} \subseteq \mathcal{J}$, let $P_{\text{sum}}(\mathcal{X}) = \sum_{J_j \in \mathcal{X}} p_j$ denote the sum of processing times of all jobs in $\mathcal{X}$, and let $W_{\text{sum}}(\mathcal{X}) = \sum_{J_j \in \mathcal{X}} w_j$ represent the total weight of these jobs. Moreover, define $p_{\max}(\mathcal{X}) = \max_{J_j \in \mathcal{X}}\{p_j\}$ as the largest processing time and $w_{\max}(\mathcal{X}) = \max_{J_j \in \mathcal{X}}\{w_j\}$ as the largest weight among the jobs in $\mathcal{X}$. For convenience, we use the simplified notations P_{sum}, W_{sum}, $p_{\max}$, and $w_{\max}$ to denote $P_{\text{sum}}(\mathcal{J})$, $W_{\text{sum}}(\mathcal{J})$, $p_{\max}(\mathcal{J})$, and $w_{\max}(\mathcal{J})$, respectively. All jobs in $\mathcal{J}$ share an unrestricted CON d (e.g., $P_{\text{sum}} \leq d$).

A solution π to this problem requires making three sequential decisions. First, one decides whether to accept or reject each job, partitioning the set $\mathcal{J}$ into an accepted set $\mathcal{A}$ and a rejected set $\mathcal{R}$. The accepted jobs are then assigned to the m machines, resulting in m disjoint subsets $\mathcal{A}_1, \mathcal{A}_2, \ldots, \mathcal{A}_m$, where $\mathcal{A}_i$ denotes the set of jobs on M_i. Finally, for each machine M_i, the processing sequence and starting times of the jobs in $\mathcal{A}_i$ should be specified.

Given a solution π for $\mathcal{J}$, let $C_j(\pi)$ denote the completion time of job $J_j \in \mathcal{A}$. J_j is classified as *early* if $J_j \in \mathcal{A}$ and $C_j(\pi) \leq d$, with its *earliness* given by $E_j(\pi) = d - C_j(\pi)$. Conversely, J_j is classified as *tardy* if $J_j \in \mathcal{A}$ and $C_j(\pi) > d$, with its *tardiness* given by

$T_j(\pi) = C_j(\pi) - d$. When the context is clear, we use the simplified notations C_j, E_j, and T_j to denote $C_j(\pi)$, $E_j(\pi)$, and $T_j(\pi)$, respectively.

The objective is to seek a solution that minimizes the integrated cost:

$$Z_{wr} = \sum_{J_j \in \mathcal{A}} w_j(E_j + T_j) + \sum_{J_j \in \mathcal{R}} r_j, \tag{1}$$

where the first term in (1) represents the TWET of accepted jobs in $\mathcal{A}$, and the second term in (1) denotes the total penalty incurred from rejected jobs in $\mathcal{R}$. Following the extended 3-field scheduling notation (Shabtay et al., 2013; Kellerer et al., 2020), we denote the problem under study by $Pm|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$.

Throughout this paper, the n jobs in $\mathcal{J}$ are assumed to be indexed such that

$$\frac{p_1}{w_1} \leq \frac{p_2}{w_2} \leq \cdots \leq \frac{p_n}{w_n}. \tag{2}$$

3 Preliminaries

In this section, we analyze several structural properties that characterize optimal solutions for problem $Pm|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$.

When $m = 1$ and the job set $\mathcal{J}$ is partitioned in advance into $\mathcal{A}$ and $\mathcal{R}$, problem $1|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$ reduces to problem $1|d_j = d, P_{\text{sum}} \leq d|\sum w_j(E_j + T_j)$ on the set $\mathcal{A}$. For $m \geq 2$, if $n \leq m$, an optimal solution simply accepts all jobs and places one on each machine, scheduling each to finish at time d; hence we restrict to $n > m$. Under this condition, we may assume without loss that each machine is nonempty, because moving a job to an idle machine and scheduling it at time d incurs no penalty. Generalizing the single-machine characterizations in Hall and Posner (1991) and Kellerer et al. (2020) to fixed subsets $\mathcal{A}_1, \mathcal{A}_2, \ldots, \mathcal{A}_m$ yields the following lemma, which describes the structure of an optimal solution on each machine. The lemma follows from standard job shifting and interchange arguments.

Lemma 3.1. *For problem $Pm|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$, there exists an optimal solution that, for each nonempty machine M_i (i.e., $\mathcal{A}_i \neq \emptyset$), $i = 1, 2, \ldots, m$, satisfies the following four features:*

(i) jobs in $\mathcal{A}_i$ are processed consecutively on M_i, while some idle time may exist prior to the first scheduled early job;

(ii) the completion time of some job in $\mathcal{A}_i$ coincides exactly with time d;

(iii) the processing order of early jobs in $\mathcal{A}_i$ follows a decreasing order of job indices;

(iv) the processing order of tardy jobs in $\mathcal{A}_i$ follows an increasing order of job indices.

Let π^* be an optimal solution for problem $Pm|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$ satisfying Lemma 3.1. To improve readability, Table 1 summarizes the notation used in Sections 3 and 5.

Table 1: Summary of key notation

Symbol	Description
$\mathcal{A}_i$	Set of accepted jobs on M_i in π^*
$\mathcal{A}_i^E$	Set of early jobs on M_i in π^*, i.e, $\mathcal{A}_i^E = \{J_j \in \mathcal{A}_i : C_j \leq d\}$
$\mathcal{A}_i^T$	Set of tardy jobs on M_i in π^*, i.e., $\mathcal{A}_i^T = \{J_j \in \mathcal{A}_i : C_j > d\}$
$\mathcal{J}_j$	Set of the first j jobs in $\mathcal{J}$, i.e., $\mathcal{J}_j = \{J_1, J_2, \ldots, J_j\}$
$\mathcal{S}_j$	Set of partial solutions for $\mathcal{J}_j$ satisfying Lemmas 3.1, 3.2, 3.3 and 3.6
π_j	Representation of $\pi_j \in \mathcal{S}_j$ as the $(2m+1)$-tuple $(e_1, \ldots, e_m, t_1, \ldots, t_m, z)$
e_i	Total processing time of early jobs on M_i in π_j
t_i	Total processing time of tardy jobs on M_i in π_j
z	Total integrated cost of π_j
Φ_j	Set of feasible tuples generated by the partial solutions in $\mathcal{S}_j$
$\mathcal{N}_j$	Set of the last $n-j+1$ jobs in $\mathcal{J}$ (i.e., $\mathcal{N}_j = \{J_j, J_{j+1}, \ldots, J_n\}$)
$\mathcal{U}_j$	Set of partial solutions for $\mathcal{N}_j$ satisfying Lemmas 3.1, 3.4 and 3.5
σ_j	Representation of $\sigma_j \in \mathcal{U}_j$ as the $(2m+1)$-tuple $(f_1, \ldots, f_m, g_1, \ldots, g_m, u)$
f_i	Total weight of early jobs on M_i in σ_j
g_i	Total weight of tardy jobs on M_i in σ_j
u	Total integrated cost of σ_j
Ψ_j	Set of feasible tuples generated by the partial solutions in $\mathcal{U}_j$

Although the four subsequent lemmas analyze tardy workloads, early workloads, tardy weights, and early weights individually, they reveal a unified balancing principle: in any optimal solution, these quantities must remain tightly balanced both across different machines and between the early and tardy sets within each machine. Specifically, Lemmas 3.2 and 3.3 bound the total processing time of $\mathcal{A}_i^T$ and $\mathcal{A}_i^E$, respectively, while Lemmas 3.4 and 3.5 bound their total weights. The proof of each lemma relies on a similar shifting argument in which any significant imbalance permits a job reassignment that strictly improves the integrated cost, thereby contradicting the assumption of optimality. The detailed proofs are provided in Appendix A.

Lemma 3.2. *Given π^* for problem $Pm|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$, we have*

$$\min_{i=1,2,\ldots,m} \{P_{\text{sum}}(\mathcal{A}_i^T)\} \leq \frac{1}{2m} P_{\text{sum}}(\mathcal{A}) \tag{3}$$

and

$$\max_{i=1,2,\ldots,m}\{P_{\text{sum}}(\mathcal{A}_i^T)\} \leq \frac{1}{2m}P_{\text{sum}}(\mathcal{A}) + p_{\max}(\mathcal{A}). \tag{4}$$

Lemma 3.3. *Given π^* for problem $Pm|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$, we have*

$$\min_{i=1,2,\ldots,m}\{P_{\text{sum}}(\mathcal{A}_i^E)\} \leq \frac{1}{2m}P_{\text{sum}}(\mathcal{A}) + p_{\max}(\mathcal{A}) \tag{5}$$

and

$$\max_{i=1,2,\ldots,m}\{P_{\text{sum}}(\mathcal{A}_i^E)\} \leq \frac{1}{2m}P_{\text{sum}}(\mathcal{A}) + 2p_{\max}(\mathcal{A}). \tag{6}$$

Lemma 3.4. *Given π^* for problem $Pm|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$, we have*

$$\min_{i=1,2,\ldots,m}\{W_{\text{sum}}(\mathcal{A}_i^T)\} \leq \frac{1}{2m}W_{\text{sum}}(\mathcal{A}) \tag{7}$$

and

$$\max_{i=1,2,\ldots,m}\{W_{\text{sum}}(\mathcal{A}_i^T)\} \leq \frac{1}{2m}W_{\text{sum}}(\mathcal{A}) + w_{\max}(\mathcal{A}). \tag{8}$$

Lemma 3.5. *Given π^* for problem $Pm|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$, we have*

$$\min_{i=1,2,\ldots,m}\{W_{\text{sum}}(\mathcal{A}_i^E)\} \leq \frac{1}{2m}W_{\text{sum}}(\mathcal{A}) + w_{\max}(\mathcal{A}) \tag{9}$$

and

$$\max_{i=1,2,\ldots,m}\{W_{\text{sum}}(\mathcal{A}_i^E)\} \leq \frac{1}{2m}W_{\text{sum}}(\mathcal{A}) + 2w_{\max}(\mathcal{A}). \tag{10}$$

The bounds established in Lemmas 3.2–3.5 impose tighter constraints on the total processing time and total weight of early and tardy jobs on each machine. Although pseudo-polynomial DP algorithms can be formulated without these restrictions, the above lemmas are essential for pruning the state space and enabling a more compact enumeration. This reduction improves the efficiency of the DP algorithms in Section 5 and enables polynomial-time algorithms for the two special cases in Section 6.

Observe that rejecting any job $J_j \in \mathcal{A}$ adds exactly r_j to the total cost and does not affect the cost of any other job. If $w_j(E_j + T_j) = r_j$, then accepting and rejecting give the same objective value; hence we may assume that such a job is rejected in some optimal solution. This yields the following result.

Lemma 3.6. *Given π^* for problem $Pm|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$, we have $w_j(E_j + T_j) < r_j$ for any job $J_j \in \mathcal{A}$.*

4 $\mathcal{NP}$-hardness proof

Since problem $1|d_j = d, P_{\text{sum}} \leq d|\sum w_j(E_j + T_j)$ is binary $\mathcal{NP}$-hard (Hall and Posner, 1991), the generalized problem $Pm|rej, d_j = d, P \leq d|Z_{wr}$ is at least binary $\mathcal{NP}$-hard,

even for fixed m. In contrast, Hall and Posner (1991) presented an $O(n)$-time algorithm for the single-machine case with proportional weights (i.e., $w_j = \theta p_j$, with θ being a positive constant). However, Sun and Wang (2003) demonstrated that this special case becomes binary $\mathcal{NP}$-hard on two machines. We strengthen these results by showing that problem $Pm|d_j = d, P_{\text{sum}} \leq d, w_j = p_j|\sum w_j(E_j + T_j)$ is strongly $\mathcal{NP}$-hard for arbitrary m, and that the rejection variant $1|rej, d_j = d, P_{\text{sum}} \leq d, w_j = p_j|Z_{wr}$ is binary $\mathcal{NP}$-hard. These results are derived via reductions from the strongly $\mathcal{NP}$-complete *3-Partition* and binary $\mathcal{NP}$-complete *Partition* problems, respectively (Garey and Johnson, 1979).

3-Partition: Given an integer b and a set $\mathcal{C} = \{c_1, c_2, \ldots, c_{3k}\}$ of $3k$ positive integers, where $\sum_{j=1}^{3k} c_j = kb$ and $b/4 < c_j < b/2$ for each $j = 1, 2, \ldots, 3k$, the problem asks whether the index set $\mathcal{I} = \{1, 2, \ldots, 3k\}$ can be partitioned into k disjoint subsets $\mathcal{I}_1, \mathcal{I}_2, \ldots, \mathcal{I}_k$ such that $\sum_{j\in\mathcal{I}_i} c_j = b$ and $|\mathcal{I}_i| = 3$ for $i = 1, 2, \ldots, k$?

Partition: Given an integer b and a set $\mathcal{C} = \{c_1, c_2, \ldots, c_k\}$ of k positive integers, where $\sum_{j=1}^{k} c_j = 2b$, the problem asks whether the index set $\mathcal{I} = \{1, 2, \ldots, k\}$ can be partitioned into two disjoint subsets $\mathcal{I}_1$ and $\mathcal{I}_2$ such that $\sum_{j\in\mathcal{I}_1} c_j = \sum_{j\in\mathcal{I}_2} c_j = b$?

To streamline the exposition of the two $\mathcal{NP}$-hardness proofs, we make extensive use of the following result.

Lemma 4.1. *For any sequence of real numbers $z_1, z_2, \ldots, z_q$, we have*

$$\sum_{j=1}^{q} z_j \sum_{i=1}^{j} z_i = \frac{1}{2}\Big(\sum_{j=1}^{q} z_j^2 + \big(\sum_{j=1}^{q} z_j\big)^2\Big).$$

Theorem 4.1. *When m is arbitrary, problem $Pm|d_j = d, P_{\text{sum}} \leq d, w_j = p_j|\sum w_j(E_j + T_j)$ is strongly $\mathcal{NP}$-hard.*

Proof. Given an instance $\mathcal{I}_{3p} = (c_1, c_2, \ldots, c_{3k}; b)$ to 3-Partition, set $\Delta = \frac{1}{2}(\sum_{j=1}^{3k} c_j^2 + kb^2)$. We build an instance $\mathcal{I}_{sch}$ to problem $Pm|d_j = d, P_{\text{sum}} \leq d, w_j = p_j|\sum w_j(E_j + T_j)$ with a set $\mathcal{M} = \{M_1, M_2, \ldots, M_m\}$ of $m = k$ machines and a set $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ of $n = 4k$ jobs. The first $3k$ jobs are referred to as *regular* jobs, where $p_j = w_j = c_j$ for $1 \leq j \leq 3k$. The remaining k jobs are called *large* jobs, where $p_{3k+j} = w_{3k+j} = \Delta + 1$ for $1 \leq j \leq k$. The common due date is $d = \sum_{j=1}^{n} p_j$ and the threshold value is $Y = \Delta$. We next show that $\mathcal{I}_{3p}$ admits a solution if and only if $\mathcal{I}_{sch}$ admits a schedule π with $\sum_{j=1}^{n} w_j(E_j(\pi) + T_j(\pi)) \leq Y$.

Assume that $\mathcal{I}_{3p}$ admits a solution $(\mathcal{I}_1, \mathcal{I}_2, \ldots, \mathcal{I}_k)$ with $\sum_{j\in\mathcal{I}_i} c_j = b$ and $|\mathcal{I}_i| = 3$ for $i = 1, 2, \ldots, k$. We construct a schedule π as follows: on M_i $(i = 1, 2, \ldots, k)$, J_{3k+i} starts its processing at time $d - \Delta - 1$, and the jobs in $\mathcal{J}_i = \{J_j : j \in \mathcal{I}_i\}$ are processed during $[d, d + b]$ in increasing index order. Hence, for each $i = 1, 2, \ldots, k$, we have $C_{3k+i}(\pi) = d$ and $C_j(\pi) = d + \sum_{l:l\leq j \text{ and } l\in\mathcal{I}_i} p_l$ for each $j \in \mathcal{I}_i$. Then, by Lemma 4.1 and straightforward calculations, we can verify that $\sum_{j=1}^{n} w_j(E_j(\pi) + T_j(\pi)) = \Delta = Y$.

Conversely, assume that $\mathcal{I}_{sch}$ admits a schedule π with $\sum_{j=1}^{n} w_j(E_j(\pi)+T_j(\pi)) \leq Y$. Since $w_{3k+i} = \Delta + 1 > Y$ for $i = 1, 2, \ldots, k$, all large jobs must be completed exactly at time d in π. This implies that $E_{3k+i}(\pi) = T_{3k+i}(\pi) = 0$ for each $i = 1, 2, \ldots, k$. Since all large jobs are identical, we can assume that J_{3k+i} is assigned to M_i and processed during $[d-\Delta-1, d]$ on M_i. Moreover, we claim that all regular jobs are tardy, i.e., $E_j(\pi) = 0$ for each $j = 1, 2, \ldots, 3k$. To see this, suppose for contradiction that some job J_x $(1 \leq x \leq 3k)$ is scheduled early on a machine M_y. This would imply $\sum_{j=1}^{n} w_j(E_j(\pi) + T_j(\pi)) \geq E_x(\pi) \geq p_{3k+y} = \Delta + 1 > Y$, contradicting the feasibility of π.

For each $i = 1, 2, \ldots, k$ and $j = 1, 2, \ldots, 3k$, we define $x_{ij} = 1$ if J_j is scheduled on M_i in π and $x_{ij} = 0$ otherwise. Since $p_j = w_j = c_j$ for $1 \leq j \leq 3k$, Lemma 3.1 allows us to assume that the tardy jobs on each machine are processed consecutively in increasing index order from time d onward in π. For each $i = 1, 2, \ldots, k$, let $\mathcal{I}_i = \{j : x_{ij} = 1, 1 \leq j \leq 3k\}$ and $\theta_i = \sum_{j \in \mathcal{I}_i} c_j$. By Lemma 4.1, for each $i = 1, 2, \ldots, k$, we have

$$\sum_{j \in \mathcal{I}_i} w_j T_j(\pi) = \sum_{j \in \mathcal{I}_i} w_j \sum_{l=1}^{j} x_{il} p_l = \sum_{j \in \mathcal{I}_i} c_j \sum_{l=1}^{j} x_{il} c_l = \frac{1}{2}(\sum_{j \in \mathcal{I}_i} c_j^2 + (\sum_{j \in \mathcal{I}_i} c_j)^2) = \frac{1}{2}(\sum_{j \in \mathcal{I}_i} c_j^2 + \theta_i^2).$$

Based on the feasibility of π and $\mathcal{I}_1 \cup \mathcal{I}_2 \cdots \cup \mathcal{I}_k = \{1, 2, \ldots, 3k\}$, we derive

$$Y \geq \sum_{j=1}^{n} w_j(E_j(\pi) + T_j(\pi)) = \sum_{j=1}^{3k} w_j T_j(\pi) = \sum_{i=1}^{k} \sum_{j \in \mathcal{I}_i} w_j T_j(\pi) = \frac{1}{2}(\sum_{j=1}^{3k} c_j^2 + \sum_{i=1}^{k} \theta_i^2). \quad (11)$$

Substituting $Y = \Delta = \frac{1}{2}(\sum_{j=1}^{3k} c_j^2 + kb^2)$ into (11), we obtain

$$\sum_{i=1}^{k} \theta_i^2 \leq kb^2. \quad (12)$$

Now, consider the following optimization problem:

$$\min \quad F(x_1, x_2, \ldots, x_k) = \sum_{i=1}^{k} x_i^2 \quad (13)$$

$$s.t. \quad \sum_{i=1}^{k} x_i = kb, \quad (14)$$

$$x_i \geq 0, \quad i = 1, 2, \ldots, k. \quad (15)$$

Since the objective function F in (13) is strictly convex and the constraint (14) is linear, the global minimum is uniquely attained when $x_1 = x_2 = \cdots = x_k = b$, yielding $F(x_1, x_2, \ldots, x_k) \geq kb^2$. Recall that $\sum_{i=1}^{k} \theta_i = \sum_{i=1}^{k} \sum_{j \in \mathcal{I}_i} c_j = \sum_{i=1}^{3k} c_j = kb$. Combining this with the upper bound in (12) and the lower bound derived above, we must have $\sum_{i=1}^{k} \theta_i^2 = kb^2$. Note that this equality holds if and only if $\theta_1 = \theta_2 = \cdots = \theta_k = b$. Given that $b/4 < c_j < b/2$ for all j, $\sum_{j \in \mathcal{I}_i} c_j = \theta_i = b$ implies that $|\mathcal{I}_i| = 3$ for each $i = 1, 2, \ldots, k$. Therefore, $(\mathcal{I}_1, \mathcal{I}_2, \ldots, \mathcal{I}_k)$ is a solution to $\mathcal{I}_{3p}$. □

Theorem 4.2. *Problem* $1|rej, d_j = d, P_{\text{sum}} \le d, w_j = p_j|Z_{wr}$ *is binary* $\mathcal{NP}$*-hard.*

Proof. Given an instance $\mathcal{I}_{par} = (c_1, c_2, \ldots, c_k; b)$ to Partition, set $\Delta = \frac{1}{2}(\sum_{j=1}^{k} c_j^2 + 3b^2)$. We build an instance $\mathcal{I}_{sch}$ to problem $1|rej, d_j = d, P_{\text{sum}} \le d, w_j = p_j|Z_{wr}$ with a set $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ of $n = k + 1$ jobs. For each $j = 1, 2, \ldots, k$, $p_j = w_j = c_j$ and $r_j = \frac{1}{2}c_j^2 + bc_j$; while $p_{k+1} = w_{k+1} = r_{k+1} = \Delta + 1$. The common due date is $d = \sum_{j=1}^{n} p_j$ and the threshold value is $Y = \Delta$. We next show that $\mathcal{I}_{par}$ admits a solution if and only if $\mathcal{I}_{sch}$ admits a solution π with $Z_{wr}(\pi) \le Y$.

Assume that $\mathcal{I}_{par}$ admits a solution $(\mathcal{I}_1, \mathcal{I}_2)$ with $\sum_{j\in\mathcal{I}_1} c_j = \sum_{j\in\mathcal{I}_2} c_j = b$. We construct a solution π to $\mathcal{I}_{sch}$ as follows: J_{k+1} is accepted and starts processing at time $d - \Delta - 1$, the jobs in $\mathcal{A}' = \{J_j : j \in \mathcal{I}_1\}$ are accepted and processed during $[d, d + b]$ in increasing index order, and the job in $\mathcal{R} = \{J_j : j \in \mathcal{I}_2\}$ are rejected. Hence, we have $C_{k+1}(\pi) = d$ and $C_j(\pi) = d + \sum_{l:l\le j \text{ and } l\in\mathcal{I}_1} p_l$ for each $J_j \in \mathcal{A}'$. Then, by Lemma 4.1 and straightforward calculations, we can verify that $Z_{wr}(\pi) = \Delta = Y$.

Conversely, assume that $\mathcal{I}_{sch}$ admits a solution π with $Z_{wr}(\pi) \le Y$. Since $r_{k+1} = w_{k+1} = \Delta + 1$, J_{k+1} must be accepted and processed during $[d - \Delta - 1, d]$ in π. Let $\mathcal{I}_1 = \{j : J_j \in \mathcal{A} \setminus \{J_{k+1}\}\} = \{j_1, j_2, \ldots, j_h\}$ and $\mathcal{I}_2 = \{j : J_j \in \mathcal{R}\}$, where $h = |\mathcal{I}_1|$ and $j_1 < j_2 < \ldots < j_h$. If any job J_x with $x \in \mathcal{I}_1$ were scheduled early, we would have $Z_{wr}(\pi) \ge E_x(\pi) \ge p_{k+1} = \Delta + 1 > Y$, a contradiction. Hence, each job J_j ($j \in \mathcal{I}_1$) must be tardy in π. Given that $p_j = w_j = c_j$ for all $j \in \mathcal{I}_1$, Lemma 3.1 allows us to assume that jobs in $\mathcal{A} \setminus \{J_{k+1}\}$ are processed consecutively in increasing index order from time d onward in π. Thus, $T_{j_g}(\pi) = C_{j_g}(\pi) - d = \sum_{i=1}^{g} p_{j_i}$ for each $g = 1, 2, \ldots, h$. By Lemma 4.1, we obtain

$$\begin{aligned}\sum_{J_j\in\mathcal{A}} w_j(E_j(\pi) + T_j(\pi)) &= \sum_{j\in\mathcal{I}_1} w_j T_j(\pi) = \sum_{g=1}^{h} w_{j_g} T_{j_g}(\pi) = \sum_{g=1}^{h} c_{j_g} \sum_{i=1}^{g} c_{j_i} \\ &= \frac{1}{2}(\sum_{g=1}^{h} c_{j_g}^2 + (\sum_{j=1}^{h} c_{j_g})^2) = \frac{1}{2}(\sum_{j\in\mathcal{I}_1} c_j^2 + (\sum_{j\in\mathcal{I}_1} c_j)^2).\end{aligned}$$

Recall that $\mathcal{I}_1 \cup \mathcal{I}_2 = \{1, 2, \ldots, k\}$ and $\sum_{j\in\mathcal{I}_1} c_j + \sum_{j\in\mathcal{I}_2} c_j = 2b$. Let $\theta = \sum_{j\in\mathcal{I}_1} c_j$. Using the feasibility of π and the relation $\sum_{J_j\in\mathcal{R}} r_j = \sum_{j\in\mathcal{I}_2} (\frac{1}{2}c_j^2 + bc_j)$, we derive

$$\begin{aligned} Y \ge Z_{wr}(\pi) &= \frac{1}{2}(\sum_{j\in\mathcal{I}_1} c_j^2 + (\sum_{j\in\mathcal{I}_1} c_j)^2) + \frac{1}{2}\sum_{j\in\mathcal{I}_2} c_j^2 + b\sum_{j\in\mathcal{I}_2} c_j \\ &= \frac{1}{2}\sum_{j=1}^{k} c_j^2 + \frac{1}{2}\theta^2 + b(2b - \theta) = \frac{1}{2}\sum_{j=1}^{k} c_j^2 + \frac{3}{2}b^2 + \frac{1}{2}(\theta - b)^2 \qquad (16) \\ &\ge \frac{1}{2}\sum_{j=1}^{k} c_j^2 + \frac{3}{2}b^2 = \Delta = Y. \end{aligned}$$

The term involving θ in (16) attains its minimum if and only if $\theta = b$. This implies that $\sum_{j\in\mathcal{I}_1} c_j = b$ and $\sum_{j\in\mathcal{I}_2} c_j = 2b - \sum_{j\in\mathcal{I}_1} c_j = b$. Therefore, $(\mathcal{I}_1, \mathcal{I}_2)$ is a solution to $\mathcal{I}_{par}$. □

5 Algorithms

In this section, we address the problem $Pm|rej, d_j = d, P_{\text{sum}} \le d|Z_{wr}$ with a fixed number of machines. We develop two pseudo-polynomial DP algorithms, $\mathbf{DP_1}$ and $\mathbf{DP_2}$, distinguished by their state representations. $\mathbf{DP_1}$ employs processing-time-based states, while $\mathbf{DP_2}$ uses weight-based states. The motivation for proposing two formulations lies in their complementary efficiency profiles. $\mathbf{DP_1}$ performs better when the total processing time is small relative to the total weight, and $\mathbf{DP_2}$ is preferable in the opposite scenario. This dual approach allows us to solve instances with diverse parameter ranges efficiently. From each exact DP algorithm, we then construct an FPTAS. Finally, we conduct computational experiments to assess the performance of two DP algorithms and their corresponding FPTASs.

To express the objective function (1) explicitly, for each $i = 1, 2, \ldots, m$ and $j = 1, 2, \ldots, n$, define the following binary variables:

- $x_{ij} = 1$ if J_j is accepted and scheduled early on M_i, and 0 otherwise;
- $y_{ij} = 1$ if J_j is accepted and scheduled tardy on M_i, and 0 otherwise;
- $z_j = 1$ if J_j is rejected, and 0 otherwise.

Since each job J_j is either accepted and scheduled (early or tardy) on some machine or rejected, the following constraints hold:

$$\sum_{i=1}^{m} x_{ij} + \sum_{i=1}^{m} y_{ij} + z_j = 1, \ j = 1, 2, \ldots, n. \tag{17}$$

Following the job indexing in (2) and applying Lemma 3.1, if J_j is accepted and scheduled early on M_i (i.e., $x_{ij} = 1$), its completion time is $C_j = d - \sum_{k=1}^{j-1} p_k x_{ik}$, and so its earliness is $\sum_{k=1}^{j-1} p_k x_{ik}$. Similarly, if J_j is accepted and scheduled tardy on M_i (i.e., $y_{ij} = 1$), its completion time is $C_j = d + \sum_{k=1}^{j} p_k y_{ik}$, and so its tardiness is $\sum_{k=1}^{j} p_k y_{ik}$. Moreover, if J_j is rejected (i.e., $z_j = 1$), define $E_j = T_j = 0$. Thus, the cost of J_j is

$$c_j = w_j\Big(\sum_{i=1}^{m} x_{ij} \sum_{k=1}^{j-1} p_k x_{ik} + \sum_{i=1}^{m} y_{ij} \sum_{k=1}^{j} p_k y_{ik}\Big) + r_j z_j. \tag{18}$$

As a consequence, the objective function (1) can be formulated as

$$Z_{wr} = \sum_{j=1}^{n} w_j\Big(\sum_{i=1}^{m} x_{ij} \sum_{k=1}^{j-1} p_k x_{ik} + \sum_{i=1}^{m} y_{ij} \sum_{k=1}^{j} p_k y_{ik}\Big) + \sum_{j=1}^{n} r_j z_j. \tag{19}$$

After rearrangement (see Appendix B for details), (19) can be rewritten as

$$Z_{wr} = \sum_{j=1}^{n} p_j \Big(\sum_{i=1}^{m} x_{ij} \sum_{k=j+1}^{n} w_k x_{ik} + \sum_{i=1}^{m} y_{ij} \sum_{k=j}^{n} w_k y_{ik}\Big) + \sum_{j=1}^{n} r_j z_j. \tag{20}$$

5.1 An exact DP algorithm

The first approach iterates over the n jobs sequentially in increasing index order, as outlined in (2). It employs processing-time-based states and computes the objective function (1) using the expression given in (19). For each $j = 1, 2, \ldots, n$, let $\mathcal{S}_j$ be the set of partial solutions that satisfy the features established in Lemmas 3.1, 3.2, 3.3 and 3.6 for $\mathcal{J}_j$. A partial solution $\pi_j \in \mathcal{S}_j$ is represented by the $(2m+1)$-tuple $(e_1, \ldots, e_m, t_1, \ldots, t_m, z)$, with e_i and t_i denoting the total processing time of early and tardy jobs on M_i, respectively, $i = 1, 2, \ldots, m$, and z being the integrated cost (i.e., the TWET of accepted jobs plus the penalty for rejected ones).

For each $j = 1, 2, \ldots, n$, let Φ_j be the set of feasible tuples generated by the partial solutions in $\mathcal{S}_j$. The algorithm starts by initializing $\Phi_0 = \{(0, 0, \ldots, 0)\}$, then dynamically builds each Φ_j based on its prior set Φ_{j-1}. We discuss the recurrence relationship between Φ_j and Φ_{j-1}. With respect to a specific tuple $(e_1, \ldots, e_m, t_1, \ldots, t_m, z) \in \Phi_j$, let $\pi_j \in \mathcal{S}_j$ be the partial solution attaining this tuple. Moreover, let π_{j-1} be the partial solution for $\mathcal{J}_{j-1}$ obtained by eliminating J_j from π_j. By Lemma 3.1, $\pi_{j-1} \in \mathcal{S}_{j-1}$. Let $(e'_1, \ldots, e'_m, t'_1, \ldots, t'_m, z')$ represent the tuple associated with π_{j-1}. To derive the tuple transition from $(e'_1, \ldots, e'_m, t'_1, \ldots, t'_m, z')$ to $(e_1, \ldots, e_m, t_1, \ldots, t_m, z)$, based on Lemma 3.1, the following $2m + 1$ distinct scenarios need to be examined.

• **Scenario i (i=1, 2, …, m):** Let J_j be accepted and scheduled early on M_i in π_j. According to Lemma 3.1, J_j is the first processed job on M_i, and its completion time is $C_j(\pi_j) = d - e_i + p_j$. As a result, $E_j(\pi_j) = e_i - p_j = e'_i$. Thus, we have $e_k = e'_k$ for $k = 1, 2, \ldots, i-1, i+1, \ldots, m$, $e_i = e'_i + p_j$, $t_l = t'_l$ for $l = 1, 2, \ldots, m$, and $z = z' + w_j e'_i$. By Lemmas 3.3 and 3.6, it suffices to consider the scenario where $e'_i + p_j \leq \frac{1}{2m} P_{\text{sum}} + 2p_{\max}$ and $w_j e'_i < r_j$.

• **Scenario m+i (i=1, 2, …, m):** Let J_j be accepted and scheduled tardy on M_i in π_j. According to Lemma 3.1, J_j is the last processed job on M_i, and its completion time is $C_j(\pi_j) = d + t_i$. As a result, $T_j(\pi_j) = t_i = t'_i + p_j$. Thus, we have $e_k = e'_k$ for $k = 1, 2, \ldots, m$, $t_l = t'_l$ for $l = 1, 2, \ldots, i-1, i+1, \ldots, m$, $t_i = t'_i + p_j$, and $z = z' + w_j(t'_i + p_j)$. By Lemmas 3.2 and 3.6, it suffices to consider the scenario where $t'_i + p_j \leq \frac{1}{2m} P_{\text{sum}} + p_{\max}$ and $w_j(t'_i + p_j) < r_j$.

• **Scenario 2m+1:** Let J_j be rejected in π_j. Since the contribution of J_j to the objective value is r_j, we have $e_k = e'_k$ for $k = 1, 2, \ldots, m$, $t_l = t'_l$ for $l = 1, 2, \ldots, m$, and $z = z' + r_j$.

The next lemma is straightforward to prove and will be employed to remove dominated tuples during the tuple generation procedure.

Lemma 5.1. *Given two partial solutions in $\mathcal{S}_j$ for $\mathcal{J}_j$ represented by tuples $(e_1, \ldots, e_m, t_1, \ldots, t_m, z)$ and $(e'_1, \ldots, e'_m, t'_1, \ldots, t'_m, z')$ in Φ_j. If $e_i \leq e'_i$ and $t_i \leq t'_i$ for each $i = 1, 2, \ldots, m$, and $z \leq z'$, then the former solution dominates the latter, implying that the tuple $(e'_1, \ldots, e'_m, t'_1, \ldots, t'_m, z')$ can be eliminated from Φ_j.*

Although Lemma 5.1 gives a general condition for eliminating dominated tuples, we do not invoke the lemma in its full generality. Instead, we use a simpler rule: when constructing Φ_j, if multiple tuples share the same e_i and t_i values for all i, keeping only the one with the smallest z value suffices for correctness. This avoids the overhead of checking the full dominance relation while preserving optimality. In light of the above discussion, we formulate the following DP algorithm (**DP$_1$**) to solve problem $Pm \mid rej, d_j = d, P_{\text{sum}} \leq d \mid Z_{wr}$.

Theorem 5.1. *Algorithm* **DP$_1$** *solves problem $Pm|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$ with a time complexity of $O(mn(\frac{1}{2m}P_{\text{sum}} + 2p_{\max})^{2m})$. When m is fixed, it is pseudo-polynomial.*

Proof. The correctness of **DP$_1$** relies on two key aspects: (i) it enumerates all feasible tuples characterizing optimal solutions that satisfy the features specified in Lemmas 3.1, 3.2, 3.3 and 3.6; and (ii) during the elimination procedure, only non-dominated tuples are preserved, as justified by Lemma 5.1.

Next, the time complexity of **DP$_1$** is estimated. Step 1 requires $O(n \log n)$ time (line 2) and Step 2 operates in linear time (line 4). Due to the bounds on e_i ($0 \leq e_i \leq \frac{1}{2m}P_{\text{sum}} + 2p_{\max}$) and t_i ($0 \leq t_i \leq \frac{1}{2m}P_{\text{sum}} + p_{\max}$), after the execution of elimination procedure (line 22), it leaves at most $O(\frac{1}{2m}P_{\text{sum}} + 2p_{\max})^{2m}$ tuples in Φ_{j-1}. Moreover, each tuple in Φ_{j-1} produces up to $2m+1$ tuples in Φ_j. Hence, Φ_j can be generated in $O(m(\frac{1}{2m}P_{\text{sum}} + 2p_{\max})^{2m})$ time (lines 8-22). Since Step 3 is iterated n times (lines 6-23) and Step 4 takes $O(\frac{1}{2m}P_{\text{sum}} + 2p_{\max})^{2m}$ time (lines 25-26), **DP$_1$** can be implemented in $O(mn(\frac{1}{2m}P_{\text{sum}} + 2p_{\max})^{2m})$ time. □

5.2 An alternative exact DP algorithm

The alternative DP approach iterates over the n jobs sequentially in decreasing index order, as outlined in (2). It uses weight-based states and computes the objective function (1) using the expression given in (20). For each $j = 1, 2, \ldots, n$, let $\mathcal{U}_j$ be the set of partial solutions that satisfy the features established in Lemmas 3.1, 3.4 and 3.5 for $\mathcal{N}_j$. A partial solution $\sigma_j \in \mathcal{U}_j$ is represented by the $(2m+1)$-tuple $(f_1, \ldots, f_m, g_1, \ldots, g_m, u)$, where f_i and g_i denote the total weight of early and tardy jobs on M_i, respectively, $i = 1, 2, \ldots, m$, and u denotes the integrated cost (i.e., the TWET of accepted jobs plus the penalty for rejected ones).

Algorithm $\mathbf{DP_1}$: An exact DP algorithm

Input: n, m, d, p_j, w_j, r_j for $j = 1, 2, \ldots, n$

[Step 1: Preprocessing]

Renumber the jobs in $\mathcal{J}$ according to (2), i.e., $\frac{p_1}{w_1} \leq \frac{p_2}{w_2} \leq \cdots \leq \frac{p_n}{w_n}$

[Step 2: Initialization]

$\Phi_0 \leftarrow \{(0, 0, \ldots, 0)\}$, $\Phi_j \leftarrow \emptyset$ for $j = 1, 2, \ldots, n$

[Step 3: Recursion]

for $j = 1$ **to** n **do**

  [Generation procedure]

  for *each tuple* $(e_1, \ldots, e_m, t_1, \ldots, t_m, z) \in \Phi_{j-1}$ **do**

    for $i = 1$ **to** m **do**

      if $e_i + p_j \leq \frac{1}{2m} P_{\text{sum}} + 2p_{\max}$ ***and*** $w_j e_i < r_j$ **then**

        Add $(e_1, \ldots, e_{i-1}, e_i + p_j, e_{i+1}, \ldots, e_m, t_1, \ldots, t_m, z + w_j e_i)$ to Φ_j

      end

    end

    for $i = 1$ **to** m **do**

      if $t_i + p_j \leq \frac{1}{2m} P_{\text{sum}} + p_{\max}$ ***and*** $w_j(t_i + p_j) < r_j$ **then**

        Add $(e_1, \ldots, e_m, t_1, \ldots, t_{i-1}, t_i + p_j, t_{i+1}, \ldots, t_m, z + w_j(t_i + p_j))$ to Φ_j

      end

    end

    Add $(e_1, \ldots, e_m, t_1, \ldots, t_m, z + r_j)$ to Φ_j

  end

  [Elimination procedure]

  Among tuples in Φ_j with identical e_i and t_i values for $i = 1, 2, \ldots, m$, keep only the tuple with minimal z value

end

[Step 4: Result]

$z^* \leftarrow \min\{z : (e_1, \ldots, e_m, t_1, \ldots, t_m, z) \in \Phi_n\}$

Recover σ^* by backtracking $(e_1^*, \ldots, e_m^*, t_1^*, \ldots, t_m^*, z^*)$

Output: Optimal solution σ^*

For each $j = 1, 2, \ldots, n$, let Ψ_j be the set of feasible tuples generated by the partial solutions in $\mathcal{U}_j$. The algorithm starts by initializing $\Psi_{n+1} = \{(0, 0, \ldots, 0)\}$, then dynamically builds each Ψ_j based on its prior set Ψ_{j+1}. We discuss the recurrence relationship between Ψ_j and Ψ_{j+1}. With respect to a specific tuple $(f_1, \ldots, f_m, g_1, \ldots, g_m, u) \in \Psi_j$, let $\sigma_j \in \mathcal{U}_j$ be the partial solution attaining this tuple. Moreover, let σ_{j+1} be the partial solution for $\mathcal{N}_{j+1}$ obtained by eliminating J_j from σ_j. By Lemma 3.1, $\sigma_{j+1} \in \mathcal{U}_{j+1}$. Let $(f'_1, \ldots, f'_m, g'_1, \ldots, g'_m, u')$ represent the tuple associated with σ_{j+1}. To derive the tuple transition from $(f'_1, \ldots, f'_m, g'_1, \ldots, g'_m, u')$ to $(f_1, \ldots, f_m, g_1, \ldots, g_m, u)$, based on Lemma 3.1, the following $2m + 1$ distinct scenarios need to be examined.

• **Scenario i (i=1, 2, ..., m):** Let J_j be accepted and scheduled early on M_i in σ_j. According to Lemma 3.1, J_j is the last processed early job on M_i. As a result, the earliness of each early job on M_i excluding J_j increases by p_j. Thus, we have $f_k = f'_k$ for $k = 1, 2, \ldots, i-1, i+1, \ldots, m$, $f_i = f'_i + w_j$, $g_l = g'_l$ for $l = 1, 2, \ldots, m$, and $u = u' + p_j f'_i$. By Lemma 3.5, it suffices to consider the scenario where $f'_i + w_j \leq \frac{1}{2m} W_{\text{sum}} + 2w_{\max}$.

• **Scenario m+i (i=1, 2, ..., m):** Let J_j be accepted and scheduled tardy on M_i in σ_j. According to Lemma 3.1, J_j is the first processed tardy job on M_i. As a result, the tardiness of each tardy job on M_i increases by p_j. Thus, we have $f_k = f'_k$ for $k = 1, 2, \ldots, m$, $g_l = g'_l$ for $l = 1, 2, \ldots, i-1, i+1, \ldots, m$, $g_i = g'_i + w_j$, and $u = u' + p_j(g'_i + w_j)$. By Lemma 3.4, it suffices to consider the scenario where $g'_i + w_j \leq \frac{1}{2m} W_{\text{sum}} + w_{\max}$.

• **Scenario $2m$+1:** Let J_j be rejected in σ_j. Since the contribution of J_j to the objective value is r_j, we have $f_k = f'_k$ for $k = 1, 2, \ldots, m$, $g_l = g'_l$ for $l = 1, 2, \ldots, m$, and $u = u' + r_j$.

Similar to Lemma 5.1, the following lemma is employed to remove dominated tuples during the tuple generation procedure.

Lemma 5.2. *Given two partial solutions in $\mathcal{U}_j$ for $\mathcal{N}_j$ represented by tuples $(f_1, \ldots, f_m, g_1, \ldots, g_m, u)$ and $(f'_1, \ldots, f'_m, g'_1, \ldots, g'_m, u')$ in Ψ_j. If $f_i \leq f'_i$ and $g_i \leq g'_i$ for each $i = 1, 2, \ldots, m$, and $u \leq u'$, then the former solution dominates the latter, implying that the tuple $(f'_1, \ldots, f'_m, g'_1, \ldots, g'_m, u')$ can be eliminated from Ψ_j.*

Although Lemma 5.2 gives a general condition for eliminating dominated tuples, we do not invoke the lemma in its full generality. Instead, we use a simpler rule: when constructing Ψ_j, if multiple tuples share the same f_i and g_i values for all i, keeping only the one with the smallest u value suffices for correctness. This avoids the overhead of checking the full dominance relation while preserving optimality. In light of the above discussion, we formulate the following DP algorithm (**DP$_2$**) to solve problem $Pm \mid rej, d_j = d, P_{\text{sum}} \leq d \mid Z_{wr}$.

Theorem 5.2. *Algorithm* **DP$_2$** *solves problem $Pm|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$ with a time complexity of $O(mn(\frac{1}{2m} W_{\text{sum}} + 2w_{\max})^{2m})$. When m is fixed, it is pseudo-polynomial.*

Algorithm $\mathbf{DP_2}$: An alternative exact DP algorithm

Input: n, m, d, p_j, w_j, r_j for $j = 1, 2, \ldots, n$

[Step 1: Preprocessing]

Renumber the jobs in $\mathcal{J}$ according to (2), i.e., $\frac{p_1}{w_1} \leq \frac{p_2}{w_2} \leq \cdots \leq \frac{p_n}{w_n}$

[Step 2: Initialization]

$\Psi_{n+1} \leftarrow \{(0, 0, \ldots, 0)\}$, $\Psi_j \leftarrow \emptyset$ for $j = n, n-1, \ldots, 1$

[Step 3: Recursion]

for $j = n$ **to** 1 **do**

  [Generation procedure]

  for *each tuple* $(f_1, \ldots, f_m, g_1, \ldots, g_m, u) \in \Psi_{j+1}$ **do**

    for $i = 1$ **to** m **do**

      if $f_i + w_j \leq \frac{1}{2m} W_{\text{sum}} + 2w_{\max}$ **then**

        Add $(f_1, \ldots, f_{i-1}, f_i + w_j, f_{i+1}, \ldots, f_m, g_1, \ldots, g_m, u + p_j f_i)$ to Ψ_j

      end

    end

    for $i = 1$ **to** m **do**

      if $g_i + w_j \leq \frac{1}{2m} W_{\text{sum}} + w_{\max}$ **then**

        Add $(f_1, \ldots, f_m, g_1, \ldots, g_{i-1}, g_i + w_j, g_{i+1} \ldots, g_m, u + p_j(g_i + w_j))$ to Ψ_j

      end

    end

    Add $(f_1, \ldots, f_m, g_1, \ldots, g_m, u + r_j)$ to Ψ_j

  end

  [Elimination procedure]

  Among tuples in Ψ_j with identical f_i and g_i values for $i = 1, 2, \ldots, m$, keep only the tuple with minimal u value

end

[Step 4: Result]

$u^* \leftarrow \min\{u : (f_1, \ldots, f_m, g_1, \ldots, g_m, u) \in \Psi_1\}$

Recover σ^* by backtracking $(f_1^*, \ldots, f_m^*, g_1^*, \ldots, g_m^*, u^*)$

Output: Optimal solution σ^*

Proof. The theorem admits a proof similar to that of Theorem 5.1. □

5.3 An FPTAS based on $\mathbf{DP_1}$

Let $\epsilon > 0$ be a rational number and set $\theta = (1+\epsilon)^{\frac{1}{n-1}}$. For any $x \geq 1$ with $\theta^k \leq x < \theta^{k+1}$, the function $\Delta(\cdot)$ is defined by $\Delta(x) = k = \lfloor \log_\theta x \rfloor$. Moreover, define $\Delta(0) = -1$.

In $\mathbf{DP_1}$, the tuple set Φ_j at iteration j has size $O(\frac{1}{2m}P_{\text{sum}} + 2p_{\max})^{2m}$, which is pseudo-polynomial. To keep the number of states polynomial, we selectively eliminate redundant states at each iteration, retaining only one among those that are sufficiently similar. This leads to $\mathbf{DP_1^\epsilon}$, an FPTAS that uses approximate DP strategy in which the tuple space is divided into boxes and only one representative tuple is kept per box. Specifically, each tuple $(e_1, \ldots, e_m, t_1, \ldots, t_m, z) \in \Phi_j^\epsilon$ is attached a label $(\Delta(e_1), \ldots, \Delta(e_m), \Delta(t_1), \ldots, \Delta(t_m))$. By construction, for each $1 \leq i \leq m$, if $e_i \geq 1$, the value $\theta^{\Delta(e_i)}$ approximates the original state variable e_i within a small multiplicative error; the same holds for $\theta^{\Delta(t_i)}$ with respect to t_i. Accordingly, if two states share the same label, one of them is discarded during the tuple elimination procedure.

The box elimination procedure in $\mathbf{DP_1^\epsilon}$ (line 22) explores geometrically growing coordinate values. Multiplicative errors may arise when each label is mapped to the smallest objective coordinates within its box for tuple elimination, instead of utilizing the label's exact value. However, the following lemma shows that the tuples remaining after the elimination procedure provide a good approximation to those generated by $\mathbf{DP_1}$.

Lemma 5.3. *For any tuple $(e_1, \ldots, e_m, t_1, \ldots, t_m, z) \in \Phi_j$ generated by Algorithm $\mathbf{DP_1}$, Algorithm $\mathbf{DP_1^\epsilon}$ delivers a corresponding tuple $(\widetilde{e_1}, \ldots, \widetilde{e_m}, \widetilde{t_1}, \ldots, \widetilde{t_m}, \widetilde{z}) \in \Phi_j^\epsilon$ satisfying (i) $\widetilde{e_i} \leq \theta^{j-1} e_i$ for $i = 1, 2, \ldots, m$, (ii) $\widetilde{t_i} \leq \theta^{j-1} t_i$ for $i = 1, 2, \ldots, m$, and (iii) $\widetilde{z} \leq \theta^{j-1} z$.*

Proof. The proof proceeds by induction on j from 1 to n. With respect to the base case $j = 1$, J_1 can be either accepted and scheduled early or tardy on any machine M_i $(i = 1, 2, \ldots, m)$, or rejected at a cost of r_1 in both $\mathbf{DP_1}$ and $\mathbf{DP_1^\epsilon}$. Since no tuple is eliminated, we have $\Phi_1^\epsilon = \Phi_1$. Thus, the statement holds trivially for $j = 1$.

By the induction hypothesis, we assume that the statement holds for $k = 1, 2, \ldots, j-1$, where $j \geq 2$. Let $(e_1, \ldots, e_m, t_1, \ldots, t_m, z) \in \Phi_j$ be a tuple generated by $\mathbf{DP_1}$ for $\mathcal{J}_j$. $\mathbf{DP_1}$ adds this tuple to Φ_j by incorporating J_j into an existing tuple $(e'_1, \ldots, e'_m, t'_1, \ldots, t'_m, z') \in \Phi_{j-1}$ for $\mathcal{J}_{j-1}$. According to the induction hypothesis, there exists a tuple $(\widetilde{e'_1}, \ldots, \widetilde{e'_m}, \widetilde{t'_1}, \ldots, \widetilde{t'_m}, \widetilde{z'}) \in \Phi_{j-1}^\epsilon$ satisfying

$$\widetilde{e'_i} \leq \theta^{j-2} e'_i,\ \widetilde{t'_i} \leq \theta^{j-2} t'_i,\ i = 1, 2, \ldots, m, \text{ and } \widetilde{z'} \leq \theta^{j-2} z'. \tag{21}$$

Algorithm $\mathbf{DP}_1^\epsilon$: An FPTAS based on $\mathbf{DP_1}$

Input: n, m, d, ϵ, p_j, w_j, r_j for $j = 1, 2, \ldots, n$

[Step 1: Preprocessing]

Renumber the jobs in $\mathcal{J}$ according to (2), i.e., $\frac{p_1}{w_1} \leq \frac{p_2}{w_2} \leq \cdots \leq \frac{p_n}{w_n}$

[Step 2: Initialization]

$\Phi_0^\epsilon \leftarrow \{(0, 0, \ldots, 0)\}$, $\Phi_j^\epsilon \leftarrow \emptyset$ for $j = 1, 2, \ldots, n$

[Step 3: Recursion]

for $j = 1$ **to** n **do**

  [Generation procedure]

  for *each tuple* $(e_1, \ldots, e_m, t_1, \ldots, t_m, z) \in \Phi_{j-1}^\epsilon$ **do**

    for $i = 1$ **to** m **do**

      if $e_i + p_j \leq (1+\epsilon)(\frac{1}{2m}P_{\text{sum}} + 2p_{\max})$ ***and*** $w_j e_i < r_j$ **then**

        Add $(e_1, \ldots, e_{i-1}, e_i + p_j, e_{i+1}, \ldots, e_m, t_1, \ldots, t_m, z + w_j e_i)$ to Φ_j^ϵ and attach its corresponding label $(\Delta(e_1), \ldots, \Delta(e_{i-1}), \Delta(e_i + p_j), \Delta(e_{i+1}), \ldots, \Delta(e_m), \Delta(t_1), \ldots, \Delta(t_m))$

      end

    end

    for $i = 1$ **to** m **do**

      if $t_i + p_j \leq (1+\epsilon)(\frac{1}{2m}P_{\text{sum}} + p_{\max})$ ***and*** $w_j(t_i + p_j) < r_j$ **then**

        Add $(e_1, \ldots, e_m, t_1, \ldots, t_{i-1}, t_i + p_j, t_{i+1}, \ldots, t_m, z + w_j(t_i + p_j))$ to Φ_j^ϵ and attach its corresponding label $(\Delta(e_1), \ldots, \Delta(e_m), \Delta(t_1), \ldots, \Delta(t_{i-1}), \Delta(t_i + p_j), \Delta(t_{i+1}), \ldots, \Delta(t_m))$

      end

    end

    Add $(e_1, \ldots, e_m, t_1, \ldots, t_m, z + r_j)$ to Φ_j^ϵ and attach its corresponding label $(\Delta(e_1), \ldots, \Delta(e_m), \Delta(t_1), \ldots, \Delta(t_m))$

  end

  [Elimination procedure]

  Among tuples in Φ_j^ϵ sharing identical labels, keep only the tuple with minimal z value, performing an arbitrary choice if the z values are identical

end

[Step 4: Result]

$\overline{z} \leftarrow \min\{z : (e_1, \ldots, e_m, t_1, \ldots, t_m, z) \in \Phi_n^\epsilon\}$

Recover $\overline{\sigma}$ by backtracking $(\overline{e_1}, \ldots, \overline{e_m}, \overline{t_1}, \ldots, \overline{t_m}, \overline{z})$

Output: Approximate solution $\overline{\sigma}$

We proceed to prove the statement for $k = j$ by examining $2m + 1$ distinct scenarios.

Scenario i (i=1, 2, ..., m): Let J_j be accepted and scheduled early on M_i in $\mathbf{DP_1}$. In this scenario, we have

$$(e_1, \ldots, e_m, t_1, \ldots, t_m, z) = (e'_1, \ldots, e'_{i-1}, e'_i + p_j, e'_{i+1}, \ldots, e'_m, t'_1, \ldots, t'_m, z' + w_j e'_i). \quad (22)$$

Recall that $(\widetilde{e'_1}, \ldots, \widetilde{e'_m}, \widetilde{t'_1}, \ldots, \widetilde{t'_m}, \widetilde{z'}) \in \Phi^{\epsilon}_{j-1}$. From the execution of $\mathbf{DP_1}$ (lines 10-12), we have $e'_i + p_j \leq \frac{1}{2m} P_{\text{sum}} + 2p_{\max}$ and $w_j e'_i < r_j$. Combining $\theta = (1+\epsilon)^{\frac{1}{n-1}}$ and (21),

$$\widetilde{e'_i} + p_j \leq \theta^{j-2} e'_i + p_j \leq \theta^{j-2}(\frac{1}{2m} P_{\text{sum}} + 2p_{\max}) \leq (1+\epsilon)(\frac{1}{2m} P_{\text{sum}} + 2p_{\max}).$$

To prove that the lemma holds in this scenario, we further distinguish two cases. First, consider the case where $w_j \widetilde{e'_i} < r_j$. From the execution of $\mathbf{DP^{\epsilon}_1}$ (lines 10-12), the tuple $(\widetilde{e'_1}, \ldots, \widetilde{e'_{i-1}}, \widetilde{e'_i} + p_j, \widetilde{e'_{i+1}}, \ldots, \widetilde{e'_m}, \widetilde{t'_1}, \ldots, \widetilde{t'_m}, \widetilde{z'} + w_j \widetilde{e'_i})$ is added to Φ^{ϵ}_j. However, since two tuples can have identical labels, this tuple may be deleted in $\mathbf{DP^{\epsilon}_1}$ (line 22) by another tuple $(\widetilde{e_1}, \ldots, \widetilde{e_m}, \widetilde{t_1}, \ldots, \widetilde{t_m}, \widetilde{z}) \in \Phi^{\epsilon}_j$. For the tuple $(\widetilde{e_1}, \ldots, \widetilde{e_m}, \widetilde{t_1}, \ldots, \widetilde{t_m}, \widetilde{z})$, according to (21), (22) and the execution of $\mathbf{DP^{\epsilon}_1}$, we have

(i) $\widetilde{e_k} \leq \theta \widetilde{e'_k} \leq \theta(\theta^{j-2} e'_k) = \theta^{j-1} e_k$, $k = 1, 2, \ldots, i-1, i+1, \ldots, m$;

(ii) $\widetilde{e_i} \leq \theta(\widetilde{e'_i} + p_j) \leq \theta(\theta^{j-2} e'_i + p_j) \leq \theta^{j-1}(e'_i + p_j) = \theta^{j-1} e_i$;

(iii) $\widetilde{t_l} \leq \theta \widetilde{t'_l} \leq \theta(\theta^{j-2} t'_l) = \theta^{j-1} t_l$, $l = 1, 2, \ldots, m$;

(iv) $\widetilde{z} \leq \widetilde{z'} + w_j \widetilde{e'_i} \leq \theta^{j-2} z' + w_j \theta^{j-2} e'_i = \theta^{j-2}(z' + w_j e'_i) \leq \theta^{j-1} z$.

Second, consider the other case where $w_j \widetilde{e'_i} \geq r_j$. From the execution of $\mathbf{DP^{\epsilon}_1}$ (line 19), the tuple $(\widetilde{e'_1}, \ldots, \widetilde{e'_m}, \widetilde{t'_1}, \ldots, \widetilde{t'_m}, \widetilde{z'} + r_j)$ is added to Φ^{ϵ}_j. However, since two tuples can have identical labels, this tuple may be deleted in $\mathbf{DP^{\epsilon}_1}$ (line 22) by another tuple $(\widetilde{e_1}, \ldots, \widetilde{e_m}, \widetilde{t_1}, \ldots, \widetilde{t_m}, \widetilde{z}) \in \Phi^{\epsilon}_j$. For the tuple $(\widetilde{e_1}, \ldots, \widetilde{e_m}, \widetilde{t_1}, \ldots, \widetilde{t_m}, \widetilde{z})$, according to (21), (22) and the execution of $\mathbf{DP^{\epsilon}_1}$, we have

(i) $\widetilde{e_k} \leq \theta \widetilde{e'_k} \leq \theta(\theta^{j-2} e'_k) = \theta^{j-1} e_k$, $k = 1, 2, \ldots, i-1, i+1, \ldots, m$;

(ii) $\widetilde{e_i} \leq \theta \widetilde{e'_i} \leq \theta(\theta^{j-2} e'_i) \leq \theta^{j-1}(e'_i + p_j) = \theta^{j-1} e_i$;

(iii) $\widetilde{t_l} \leq \theta \widetilde{t'_l} \leq \theta(\theta^{j-2} t'_l) = \theta^{j-1} t_l$, $l = 1, 2, \ldots, m$;

(iv) $\widetilde{z} \leq \widetilde{z'} + r_j \leq \widetilde{z'} + w_j \widetilde{e'_i} \leq \theta^{j-2} z' + w_j \theta^{j-2} e'_i = \theta^{j-2}(z' + w_j e'_i) \leq \theta^{j-1} z$.

This establishes that the statement holds for $k = j$ when J_j is accepted and scheduled early on M_i in $\mathbf{DP_1}$.

Scenario m+i (i=1, 2, ..., m): Let J_j be accepted and scheduled tardy on M_i in $\mathbf{DP_1}$. In this scenario, we have

$$(e_1, \ldots, e_m, t_1, \ldots, t_m, z) = (e'_1, \ldots, e'_m, t'_1, \ldots, t'_{i-1}, t'_i + p_j, t'_{i+1}, \ldots, t'_m, z' + w_j(t'_i + p_j)). \quad (23)$$

Recall that $(\widetilde{e'_1},\ldots,\widetilde{e'_m},\widetilde{t'_1},\ldots,\widetilde{t'_m},\widetilde{z'}) \in \Phi^{\epsilon}_{j-1}$. From the execution of $\mathbf{DP_1}$ (lines 15-17), we have $t'_i + p_j \leq \frac{1}{2m}P_{\text{sum}} + p_{\max}$ and $w_j(t'_i + p_j) < r_j$. Combining $\theta = (1+\epsilon)^{\frac{1}{n-1}}$ and (21),

$$\widetilde{t'_i} + p_j \leq \theta^{j-2}t'_i + p_j \leq \theta^{j-2}(\frac{1}{2m}P_{\text{sum}} + p_{\max}) \leq (1+\epsilon)(\frac{1}{2m}P_{\text{sum}} + p_{\max}).$$

To prove that the lemma holds in this scenario, we further distinguish two cases. First, consider the case where $w_j(\widetilde{t'_i} + p_j) < r_j$. From the execution of $\mathbf{DP^{\epsilon}_1}$ (lines 15-17), the tuple $(\widetilde{e'_1},\ldots,\widetilde{e'_m},\widetilde{t'_1},\ldots,\widetilde{t'_{i-1}},\widetilde{t'_i + p_j},\widetilde{t'_{i+1}},\ldots,\widetilde{t'_m},\widetilde{z'} + w_j(\widetilde{t'_i} + p_j))$ is added to Φ^{ϵ}_j. However, since two tuples can have identical labels, this tuple may be deleted in $\mathbf{DP^{\epsilon}_1}$ (line 22) by another tuple $(\widetilde{e_1},\ldots,\widetilde{e_m},\widetilde{t_1},\ldots,\widetilde{t_m},\widetilde{z}) \in \Phi^{\epsilon}_j$. For the tuple $(\widetilde{e_1},\ldots,\widetilde{e_m},\widetilde{t_1},\ldots,\widetilde{t_m},\widetilde{z})$, according to (21), (23) and the execution of $\mathbf{DP^{\epsilon}_1}$, we have

(i) $\widetilde{e_k} \leq \theta\widetilde{e'_k} \leq \theta(\theta^{j-2}e'_k) = \theta^{j-1}e_k$, $k = 1,2,\ldots,m$;

(ii) $\widetilde{t_l} \leq \theta\widetilde{t'_l} \leq \theta(\theta^{j-2}t'_l) = \theta^{j-1}t_l$, $l = 1,2,\ldots,i-1,i+1,\ldots,m$;

(iii) $\widetilde{t_i} \leq \theta(\widetilde{t'_i} + p_j) \leq \theta(\theta^{j-2}t'_i + p_j) \leq \theta^{j-1}(t'_i + p_j) = \theta^{j-1}t_i$;

(iv) $\widetilde{z} \leq \widetilde{z'} + w_j(\widetilde{t'_i} + p_j) \leq \theta^{j-2}z' + w_j(\theta^{j-2}t'_i + p_j) \leq \theta^{j-2}(z' + w_j(t'_i + p_j)) \leq \theta^{j-1}z$.

Second, consider the other case where $w_j(\widetilde{t'_i} + p_j) \geq r_j$. From the execution of $\mathbf{DP^{\epsilon}_1}$ (line 19), the tuple $(\widetilde{e'_1},\ldots,\widetilde{e'_m},\widetilde{t'_1},\ldots,\widetilde{t'_m},\widetilde{z'} + r_j)$ is added to Φ^{ϵ}_j. However, since two tuples can have identical labels, this tuple may be deleted in $\mathbf{DP^{\epsilon}_1}$ (line 22) by another tuple $(\widetilde{e_1},\ldots,\widetilde{e_m},\widetilde{t_1},\ldots,\widetilde{t_m},\widetilde{z}) \in \Phi^{\epsilon}_j$. For the tuple $(\widetilde{e_1},\ldots,\widetilde{e_m},\widetilde{t_1},\ldots,\widetilde{t_m},\widetilde{z})$, according to (21), (23) and the execution of $\mathbf{DP^{\epsilon}_1}$, we have

(i) $\widetilde{e_k} \leq \theta\widetilde{e'_k} \leq \theta(\theta^{j-2}e'_k) = \theta^{j-1}e_k$, $k = 1,2,\ldots,m$;

(ii) $\widetilde{t_l} \leq \theta\widetilde{t'_l} \leq \theta(\theta^{j-2}t'_l) = \theta^{j-1}t_l$, $l = 1,2,\ldots,i-1,i+1,\ldots,m$;

(iii) $\widetilde{t_i} \leq \theta\widetilde{t'_i} \leq \theta(\theta^{j-2}t'_i) < \theta^{j-1}(t'_i + p_j) = \theta^{j-1}t_i$;

(iv) $\widetilde{z} \leq \widetilde{z'} + r_j \leq \widetilde{z'} + w_j(\widetilde{t'_i} + p_j) \leq \theta^{j-2}z' + w_j(\theta^{j-2}t'_i + p_j) \leq \theta^{j-2}(z' + w_j(t'_i + p_j)) \leq \theta^{j-1}z$.

This establishes that the statement holds for $k = j$ when J_j is accepted and scheduled tardy on M_i in $\mathbf{DP_1}$.

Scenario $2m+1$: Let J_j be rejected in $\mathbf{DP_1}$. In this scenario, we have

$$(e_1,\ldots,e_m,t_1,\ldots,t_m,z) = (e'_1,\ldots,e'_m,t'_1,\ldots,t'_m,z' + r_j). \tag{24}$$

Since $(\widetilde{e'_1},\ldots,\widetilde{e'_m},\widetilde{t'_1},\ldots,\widetilde{t'_m},\widetilde{z'}) \in \Phi^{\epsilon}_{j-1}$, from the execution of $\mathbf{DP^{\epsilon}_1}$ (line 19), the tuple $(\widetilde{e'_1},\ldots,\widetilde{e'_m},\widetilde{t'_1},\ldots,\widetilde{t'_m},\widetilde{z'} + r_j)$ is added to Φ^{ϵ}_j. However, since two tuples can have identical labels, this tuple may be deleted in $\mathbf{DP^{\epsilon}_1}$ (line 22) by another tuple $(\widetilde{e_1},\ldots,\widetilde{e_m},\widetilde{t_1},\ldots,\widetilde{t_m},\widetilde{z}) \in \Phi^{\epsilon}_j$. For the tuple $(\widetilde{e_1},\ldots,\widetilde{e_m},\widetilde{t_1},\ldots,\widetilde{t_m},\widetilde{z})$, according to (21), (24) and the execution of $\mathbf{DP^{\epsilon}_1}$, we have

(i) $\widetilde{e_k} \leq \theta\widetilde{e'_k} \leq \theta(\theta^{j-2}e'_k) = \theta^{j-1}e_k$, $k = 1,2,\ldots,m$;

(ii) $\widetilde{t}_l \le \theta\widetilde{t}'_l \le \theta(\theta^{j-2}t'_l) = \theta^{j-1}t_l$, $l = 1, 2, \ldots, m$;

(iii) $\widetilde{z} \le \widetilde{z'} + r_j \le \theta^{j-2}z' + r_j \le \theta^{j-2}(z' + r_j) \le \theta^{j-1}z$.

This establishes that the statement holds for $k = j$ when J_j is rejected in $\mathbf{DP_1}$. Thus, the induction hypothesis holds for $k = j$ in all scenarios, the lemma is proved. □

To estimate the time complexity of $\mathbf{DP_1^\epsilon}$, we utilize the following well-known result, which follows from the fact that $f(z) = z\ln z - (z-1)$ is increasing and satisfies $f(1) = 0$.

Lemma 5.4. *For any real $z \ge 1$, the inequality $\ln z \ge \frac{z-1}{z}$ holds.*

Theorem 5.3. *When m is fixed, Algorithm $\mathbf{DP_1^\epsilon}$ is an FPTAS for problem $Pm|rej, d_j = d, P_{\text{sum}} \le d|Z_{wr}$ with a time complexity of $O(\frac{n^{2m+1}}{\epsilon^{2m}}(\log P_{\text{sum}})^{2m})$.*

Proof. We begin by estimating the cost of the solution $\overline{\sigma}$ produced by $\mathbf{DP_1^\epsilon}$. By Theorem 5.1, an optimal solution σ^* can be produced by $\mathbf{DP_1}$. Let $(e_1^*, \ldots, e_m^*, t_1^*, \ldots, t_m^*, z^*) \in \Phi_n$ be the tuple corresponding to σ^*. Based on Lemma 5.3, there exists a tuple $(\widetilde{e_1}, \ldots, \widetilde{e_m}, \widetilde{t_1}, \ldots, \widetilde{t_m}, \widetilde{z}) \in \Phi_n^\epsilon$ with $\widetilde{z} \le \theta^{n-1}z^*$. Since $\theta = (1+\epsilon)^{\frac{1}{n-1}}$, we have $\widetilde{z} \le \theta^{n-1}z^* = (1+\epsilon)z^*$. Thus, $\mathbf{DP_1^\epsilon}$ (lines 25-26) delivers an approximate solution $\overline{\sigma}$ satisfying $\overline{z} \le \widetilde{z} \le (1+\epsilon)z^*$.

Next, the time complexity of $\mathbf{DP_1^\epsilon}$ is estimated. Step 1 requires $O(n\log n)$ time (line 2) and Step 2 operates in linear time (line 4). In Step 3, due to the elimination procedure, at most one tuple is preserved for each possible label (line 22). Since each tuple generates at most $2m+1$ new tuples, the total number of preserved tuples cannot exceed $2m+1$ times the number of labels. With respect to each tuple $(e_1, \ldots, e_m, t_1, \ldots, t_m, z) \in \Phi_j^\epsilon$, its corresponding label is of the form $(\Delta(e_1), \ldots, \Delta(e_m), \Delta(t_1), \ldots, \Delta(t_m))$, where $e_i \le \min\{(1+\epsilon)(\frac{1}{2m}P_{\text{sum}} + 2p_{\max}), P_{\text{sum}}\} \le P_{\text{sum}}$ and $t_i \le \min\{(1+\epsilon)(\frac{1}{2m}P_{\text{sum}} + p_{\max}), P_{\text{sum}}\} \le P_{\text{sum}}$ for each $i = 1, 2, \ldots, m$. By Lemma 5.4 and noting that $\theta = (1+\epsilon)^{\frac{1}{n-1}}$, the number of distinct values of $\Delta(e_i)$ is bounded by $1 + \lfloor \log_\theta P_{\text{sum}} \rfloor = 1 + \lfloor \frac{\ln P_{\text{sum}}}{\ln \theta} \rfloor = O(\frac{n\log P_{\text{sum}}}{\epsilon})$. Similarly, the number of distinct values of $\Delta(t_i)$ is also bounded by $O(\frac{n\log P_{\text{sum}}}{\epsilon})$. Hence, after the elimination procedure, the number of tuples in Φ_j^ϵ is $O(\frac{n^{2m}}{\epsilon^{2m}}(\log P_{\text{sum}})^{2m})$. Since Step 3 is iterated n times (lines 6-23) and Step 4 takes $O(\frac{n^{2m}}{\epsilon^{2m}}(\log P_{\text{sum}})^{2m})$ time (lines 25-26), $\mathbf{DP_1^\epsilon}$ can be implemented in $O(\frac{n^{2m+1}}{\epsilon^{2m}}(\log P_{\text{sum}})^{2m})$ time. □

To illustrate the implementation of $\mathbf{DP_1}$ and $\mathbf{DP_1^\epsilon}$, we provide a numerical example for problem $1|rej, d_j = d, P_{\text{sum}} \le d|Z_{wr}$. The instance consists of three jobs with parameters $(p_1, w_1, r_1) = (6, 9, 38)$, $(p_2, w_2, r_2) = (7, 7, 40)$, and $(p_3, w_3, r_3) = (7, 5, 50)$.

Using $\mathbf{DP_1}$, the tuple generation process is depicted in Figure 1, and the exact computations are given in Appendix C. The optimal solution value is $z^* = 70$, corresponding to

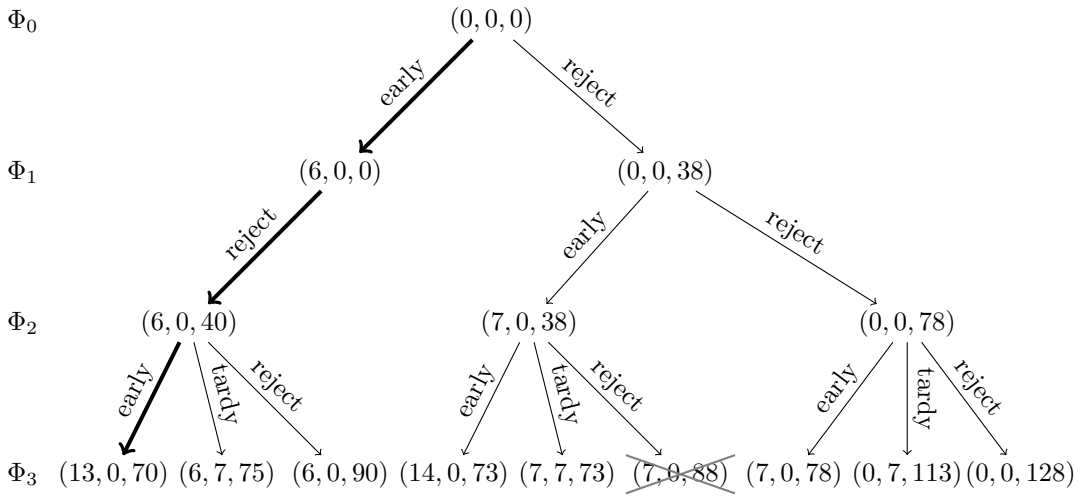

Figure 1: The generation process of $\mathbf{DP_1}$

the tuple $(13, 0, 70)$ in Φ_3. This solution is achieved by accepting jobs J_1 and J_3 (both early, with $J_3 \prec J_1$) and rejecting job J_2. The bold lines in Figure 1 represent the optimal tuple transition process.

For the same instance, we execute $\mathbf{DP_1^\epsilon}$ with $\epsilon = 1$. Here, $\theta = (1+\epsilon)^{\frac{1}{n-1}} = \sqrt{2} \approx 1.4142$, $\Delta(0) = -1$, and $\Delta(x) = \lfloor \log_\theta x \rfloor = \left\lfloor \frac{\log x}{\log \theta} \right\rfloor$ for $x \geq 1$. Since $\Delta(6) = \Delta(7) = 5$, the two tuples $(6, 0, 40)$ and $(7, 0, 38)$ in Φ_2 share the same label. Because $38 < 40$, the tuple $(6, 0, 40)$ is eliminated. Similarly, $(7, 0, 88)$ and $(7, 0, 78)$ in Φ_3 have the same label, and with $78 < 88$, the tuple $(7, 0, 88)$ is eliminated. The resulting tuple generation process is illustrated in Figure 2. The approximate solution value is $\overline{z} = 73$, which corresponds to two tuples $(14, 0, 73)$ and $(7, 7, 73)$ in Φ_3^ϵ. The first is achieved by accepting jobs J_2 and J_3 (both early, with $J_3 \prec J_2$) and rejecting job J_1; the second by accepting J_2 early and J_3 tardy, while rejecting job J_1.

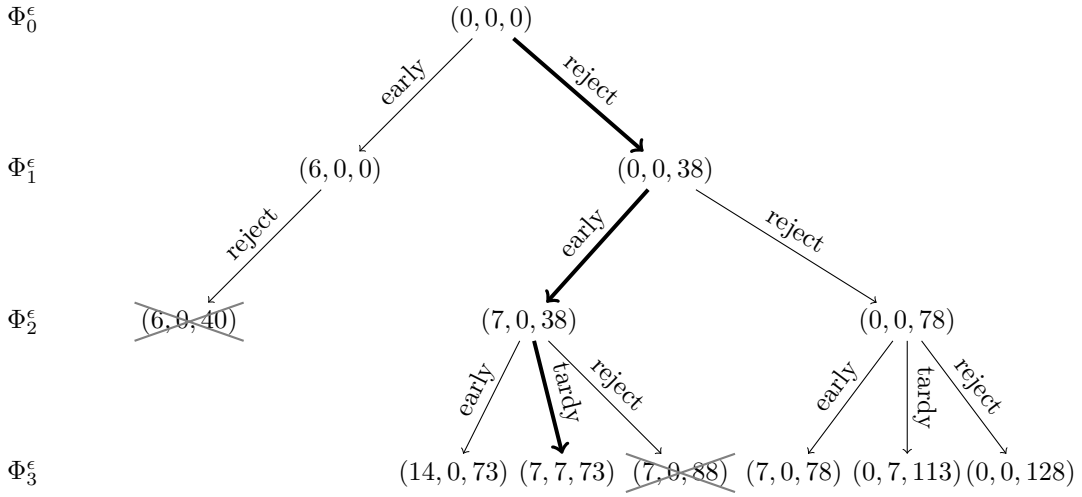

Figure 2: The generation process of $\mathbf{DP_1^\epsilon}$

5.4 An FPTAS based on $\mathbf{DP_2}$

Similar to $\mathbf{DP_1^\epsilon}$, the following optimization algorithm (referred to as $\mathbf{DP_2^\epsilon}$) can convert $\mathbf{DP_2}$ into an FPTAS.

Lemma 5.5. *For any tuple $(f_1, \ldots, f_m, g_1, \ldots, g_m, u) \in \Psi_j$ generated by Algorithm $\mathbf{DP_2}$, Algorithm $\mathbf{DP_2^\epsilon}$ delivers a corresponding tuple $(\widetilde{f_1}, \ldots, \widetilde{f_m}, \widetilde{g_1}, \ldots, \widetilde{g_m}, \widetilde{u}) \in \Psi_j^\epsilon$ satisfying (i) $\widetilde{f_i} \leq \theta^{n-j} f_i$ for $i = 1, 2, \ldots, m$, (ii) $\widetilde{g_i} \leq \theta^{n-j} g_i$ for $i = 1, 2, \ldots, m$, and (iii) $\widetilde{u} \leq \theta^{n-j} z$.*

Proof. The lemma admits a proof similar to that of Lemma 5.3. □

Theorem 5.4. *When m is fixed, Algorithm $\mathbf{DP_2^\epsilon}$ is an FPTAS for problem $Pm|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$ with a time complexity of $O(\frac{n^{2m+1}}{\epsilon^{2m}}(\log W_{\text{sum}})^{2m})$.*

Proof. The theorem admits a proof similar to that of Theorem 5.3. □

5.5 Numerical study

To evaluate and compare the computational efficiency of the exact DP algorithms and their FPTAS counterparts developed in Sections 5.1-5.4, we perform a series of computational experiments on the single-machine problem $1|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$. These experiments are not intended as a full-scale evaluation of the parallel-machine case; rather, they serve to illustrate the growth of the DP state space and the practical behavior of the FPTAS. For this reason, we restrict our attention to the single-machine setting, where these algorithmic insights can be most clearly demonstrated. All codes were implemented in Matlab and run on a PC with 12th Gen Intel(R) i7-1260P CPU (2.10 GHz) and 16 GB of memory.

The experiments are divided into three groups, each using controlled parameter settings to ensure a fair and systematic comparison. Let $\mathcal{U}\{a, b\}$ denote a discrete uniform distribution over the integers $\{a, a+1, \ldots, b\}$. For each group, problem instances are generated randomly according to the specified uniform distributions. Since the rejection cost does not affect algorithm performance, it is set to maintain a meaningful trade-off between acceptance and rejection. Specifically, $r_j \sim \mathcal{U}\{1, r_{\max}\}$ with $r_{\max} = nw_{\max}p_{\max}/10$.

Group 1 examines the performance of $\mathbf{DP_1}$ and $\mathbf{DP_2}$ across a broad range of problem sizes. Here, $n \in \{40, 60, 80, 100, 120, 140, 200\}$, $p_j \sim \mathcal{U}\{1, p_{\max}\}$, and $w_j \sim \mathcal{U}\{1, w_{\max}\}$, with $p_{\max} \in \{20, 60\}$ and $w_{\max} \in \{20, 60\}$.

Group 2 compares $\mathbf{DP_1}$ with its FPTAS counterpart $\mathbf{DP_1^\epsilon}$. To avoid the scenario where the FPTAS generates more states than the exact DP (e.g., the relative error ϵ is small and processing times are not large), we set $\epsilon \in \{0.2, 0.5, 1.0\}$. In this group, $n \in \{40, 60, 80, 100, 120\}$, $p_j \sim \mathcal{U}\{1, p_{\max}\}$ with $p_{\max} \in \{100, 200\}$, and $w_j \sim \mathcal{U}\{1, 50\}$.

Algorithm $\mathbf{DP}_{\mathbf{2}}^{\epsilon}$: An FPTAS based on $\mathbf{DP_2}$

Input: n, m, d, ϵ, p_j, w_j, r_j for $j = 1, 2, \ldots, n$

[Step 1: Preprocessing]

Renumber the jobs in $\mathcal{J}$ according to (2), i.e., $\frac{p_1}{w_1} \leq \frac{p_2}{w_2} \leq \cdots \leq \frac{p_n}{w_n}$

[Step 2: Initialization]

$\Psi_{n+1}^{\epsilon} \leftarrow \{(0, 0, \ldots, 0)\}$, $\Psi_j^{\epsilon} \leftarrow \emptyset$ for $j = n, n-1, \ldots, 1$

[Step 3: Recursion]

for $j = n$ **to** 1 **do**

  [Generation procedure]

  for *each tuple* $(f_1, \ldots, f_m, g_1, \ldots, g_m, u) \in \Psi_{j+1}^{\epsilon}$ **do**

    for $i = 1$ **to** m **do**

      if $f_i + w_j \leq (1+\epsilon)(\frac{W_{\text{sum}}}{2m} + 2w_{\max})$ **then**

        Add $(f_1, \ldots, f_{i-1}, f_i + w_j, f_{i+1}, \ldots, f_m, g_1, \ldots, g_m, u + p_j f_i)$ to Ψ_j^{ϵ} and attach its corresponding label $(\Delta(f_1), \ldots, \Delta(f_{i-1}), \Delta(f_i + w_j), \Delta(f_{i+1}), \ldots, \Delta(f_m), \Delta(g_1), \ldots, \Delta(g_m))$

      end

    end

    for $i = 1$ **to** m **do**

      if $g_i + w_j \leq (1+\epsilon)(\frac{W_{\text{sum}}}{2m} + w_{\max})$ **then**

        Add $(f_1, \ldots, f_m, g_1, \ldots, g_{i-1}, g_i + w_j, g_{i+1} \ldots, g_m, u + p_j(g_i + w_j))$ to Ψ_j^{ϵ} and attach its corresponding label $(\Delta(f_1), \ldots, \Delta(f_m), \Delta(g_1), \ldots, \Delta(g_{i-1}), \Delta(g_i + w_j), \Delta(g_{i+1}), \ldots, \Delta(g_m))$

      end

    end

    Add $(f_1, \ldots, f_m, g_1, \ldots, g_m, u + r_j)$ to Ψ_j^{ϵ} and attach its corresponding label $(\Delta(f_1), \ldots, \Delta(f_m), \Delta(g_1), \ldots, \Delta(g_m))$

  end

  [Elimination procedure]

  Among tuples in Ψ_j^{ϵ} sharing identical labels, keep only the tuple with minimal u value, performing an arbitrary choice if the u values are identical

end

[Step 4: Result]

$\overline{u} \leftarrow \min\{u : (f_1, \ldots, f_m, g_1, \ldots, g_m, u) \in \Psi_1^{\epsilon}\}$

Recover $\overline{\sigma}$ by backtracking $(\overline{f_1}, \ldots, \overline{f_m}, \overline{g_1}, \ldots, \overline{g_m}, \overline{u})$

Output: Approximate solution $\overline{\sigma}$

Group 3 compares $\mathbf{DP_2}$ with $\mathbf{DP_2^\epsilon}$. Following the same reason, $n \in \{40, 60, 80, 100, 120\}$, $p_j \sim \mathcal{U}\{1, 50\}$, $w_j \sim \mathcal{U}\{1, w_{\max}\}$ with $w_{\max} \in \{100, 200\}$, and $\epsilon \in \{0.2, 0.5, 1.0\}$.

For each specific setting of n, $p_{\max}$, $w_{\max}$, and ϵ, we generate and solve 30 random instances. The average (Avg) and maximum (Max) running times (in seconds) are reported in Tables 2–4. Entries marked with "–" signify instances that exceeded the 5400-second limit.

Table 2: Running times (in seconds) of $\mathbf{DP_1}$ and $\mathbf{DP_2}$ for $1|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$

	$(p_{\max}, w_{\max}) = (20, 20)$				$(p_{\max}, w_{\max}) = (20, 60)$				$(p_{\max}, w_{\max}) = (60, 20)$				$(p_{\max}, w_{\max}) = (60, 60)$			
	$\mathbf{DP_1}$		$\mathbf{DP_2}$		$\mathbf{DP_1}$		$\mathbf{DP_2}$		$\mathbf{DP_1}$		$\mathbf{DP_2}$		$\mathbf{DP_1}$		$\mathbf{DP_2}$	
n	Avg	Max	Avg	Max	Avg	Max	Avg	Max	Avg	Max	Avg	Max	Avg	Max	Avg	Max
40	0.04	0.06	0.05	0.07	0.04	0.06	0.59	0.88	0.61	0.92	0.04	0.06	0.60	0.86	0.60	0.99
60	0.17	0.27	0.17	0.25	0.18	0.29	2.12	2.75	2.13	3.39	0.18	0.32	2.07	2.92	2.16	3.04
80	0.57	0.80	0.53	0.77	0.61	0.82	5.72	7.47	6.00	7.92	0.58	0.80	6.09	7.81	6.16	7.87
100	1.07	1.37	1.01	1.38	1.09	1.35	10.63	14.16	10.80	14.29	1.05	1.28	10.13	14.85	10.66	14.78
120	1.84	2.34	1.81	2.42	1.87	2.34	23.96	46.37	23.24	45.28	1.93	2.46	20.95	48.18	21.97	54.32
140	3.02	3.92	2.98	3.95	2.99	3.79	–	–	–	–	2.94	3.83	–	–	–	–
200	9.25	11.33	8.96	11.29	9.24	10.78	–	–	–	–	9.18	11.31	–	–	–	–

Table 3: Running times (in seconds) of $\mathbf{DP_1}$ and $\mathbf{DP_1^\epsilon}$ for $1|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$

	$p_{\max} = 100$						$p_{\max} = 200$							
	$\mathbf{DP_1}$		$\mathbf{DP_1^\epsilon}$				$\mathbf{DP_1}$		$\mathbf{DP_1^\epsilon}$					
			$\epsilon = 0.5$		$\epsilon = 1.0$				$\epsilon = 0.2$		$\epsilon = 0.5$		$\epsilon = 1.0$	
n	Avg	Max	Avg	Max	Avg	Max	Avg	Max	Avg	Max	Avg	Max	Avg	Max
40	1.84	2.62	0.61	0.67	0.21	0.26	7.53	13.41	3.63	3.91	0.78	0.86	0.26	0.29
60	6.58	8.79	2.56	2.91	0.92	1.03	1664.51	–	15.21	18.39	3.36	3.65	1.20	1.31
80	19.84	28.29	7.58	8.41	2.69	3.16	–	–	–	–	9.76	10.44	3.34	3.56
100	–	–	18.75	19.31	5.30	5.84	–	–	–	–	24.31	27.25	6.82	7.55
120	–	–	–	–	9.53	10.46	–	–	–	–	–	–	12.62	13.98

As shown in Table 2, both $\mathbf{DP}_1$ and $\mathbf{DP}_2$ demonstrate increasing computational demand as n grows, with $\mathbf{DP}_1$ being sensitive to $p_{\max}$ and $\mathbf{DP}_2$ to $w_{\max}$. For small parameter values ($p_{\max} = 20$ and $w_{\max} = 20$), both algorithms show similar efficiency, processing 200 jobs within 12 seconds. However, significant parameter differences reveal distinct performance advantages. With $n = 140$, $p_{\max} = 20$, and $w_{\max} = 60$, $\mathbf{DP}_1$ completes in 4 seconds, while $\mathbf{DP}_2$ fails to complete within the 5400-second limit. Conversely, when $n = 140$, $p_{\max} = 60$,

Table 4: Running times (in seconds) of $\mathbf{DP_2}$ and $\mathbf{DP_2^\epsilon}$ for $1|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$

	$w_{\max} = 100$						$w_{\max} = 200$							
	$\mathbf{DP_2}$		$\mathbf{DP_2^\epsilon}$				$\mathbf{DP_2}$		$\mathbf{DP_2^\epsilon}$					
			$\epsilon = 0.5$		$\epsilon = 1.0$				$\epsilon = 0.2$		$\epsilon = 0.5$		$\epsilon = 1.0$	
n	Avg	Max	Avg	Max	Avg	Max	Avg	Max	Avg	Max	Avg	Max	Avg	Max
40	1.78	2.32	0.64	0.70	0.22	0.26	6.89	12.31	3.49	3.78	0.83	0.92	0.28	0.34
60	6.44	9.18	2.63	2.93	0.94	1.05	1521.67	–	18.81	20.65	3.30	3.64	1.19	1.33
80	19.87	36.29	7.33	8.38	2.62	2.94	–	–	–	–	9.91	10.66	3.35	3.64
100	–	–	18.34	19.33	5.32	5.81	–	–	–	–	25.89	30.63	7.01	7.52
120	–	–	–	–	9.41	10.28	–	–	–	–	–	–	12.83	14.70

and $w_{\max} = 20$, $\mathbf{DP}_2$ remains efficient (≤ 4 seconds), whereas $\mathbf{DP}_1$ exceeds the time limit. For large parameter values ($p_{\max} = 60$ and $w_{\max} = 60$), both algorithms solve all instances with $n \leq 120$ within 55 seconds, but fail to solve the 140-job case within the time limit.

The results in Tables 3 and 4 show that the FPTAS variants are substantially faster than their exact counterparts. Both $\mathbf{DP_1^\epsilon}$ and $\mathbf{DP_2^\epsilon}$ exhibit better scalability with respect to n and the relevant job parameters. Even in instances where the exact DP algorithm exceeds the time limit, the corresponding FPTAS remains practically viable. For instance, at $n = 80$ with $p_{\max} = 200$ (or $w_{\max} = 200$), $\mathbf{DP_1^\epsilon}$ (or $\mathbf{DP_2^\epsilon}$) finishes within 11 seconds. Consistent with expectations, increasing ϵ from 0.2 to 1.0 yields considerably shorter running times. Moreover, the solutions obtained by the FPTASs are either optimal or have a very small optimality gap. For instance, approximately 0.02% gap for $n = 60$, $\epsilon = 0.5$ with $p_{\max} = 100$ for $\mathbf{DP_1^\epsilon}$, and similarly with $w_{\max} = 100$ for $\mathbf{DP_2^\epsilon}$.

6 Two special cases

In this section, we design $O(n^2)$-time algorithms for two special cases of problem $Pm|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$. Building on the result from Hardy et al. (1967), the following lemma facilitates establishing the optimal job schedule for any given $\mathcal{A}$.

Lemma 6.1. *Given two sequences of real numbers $(x_1, x_2, \ldots, x_n)$ and $(y_1, y_2, \ldots, y_n)$, the minimum value of the sum $\sum x_j y_j$ is achieved when one sequence is monotonically increasing and the other is monotonically decreasing.*

6.1 $Pm|rej, d_j = d, P_{\text{sum}} \le d, p_j = p|Z_{wr}$

We address the case where all jobs share identical processing times. This case can be denoted by $Pm|rej, d_j = d, P_{\text{sum}} \le d, p_j = p|Z_{wr}$. The identical-processing-time scheduling is prevalent in repetitive manufacturing settings, especially for mass production of standardized or similar products (Fomin and Goldengorin, 2022; Fowler and Monch, 2022).

Utilizing the results of Lemmas 3.1, 3.2 and 3.3, for each machine M_i $(i = 1, 2, \ldots, m)$, the possible completion times of early jobs on M_i are $d - (\lfloor \frac{n}{2m} \rfloor + 1)p, d - \lfloor \frac{n}{2m} \rfloor p, \ldots, d$, and the possible completion times of tardy jobs on M_i are $d + p, d + 2p, \ldots, d + (\lfloor \frac{n}{2m} \rfloor + 1)p$. Hence, the possible earliness of early jobs on M_i are $(\lfloor \frac{n}{2m} \rfloor + 1)p, \lfloor \frac{n}{2m} \rfloor p, \ldots, 0$, and the possible tardiness of tardy jobs on M_i are $p, 2p, \ldots, (\lfloor \frac{n}{2m} \rfloor + 1)p$. Hence, to minimize the objective function (1), we only need to select the n smallest earliness and tardiness costs from the m machines. Let $\mathcal{C} = \{c_1, c_2, \ldots, c_n\}$ be the set of these n smallest costs, and assume without loss of generality that $c_1 \le c_2 \le \cdots \le c_n$. Then we have $c_1 = c_2 = \cdots = c_m = 0$, $c_{m+1} = c_{m+2} = \cdots = c_{3m} = p$, $\cdots$, $c_{n-v} = c_{n-v+1} = \cdots = c_n = (\lfloor \frac{n-m}{2m} \rfloor + 1)p$, where $v = n - m - 2m\lfloor \frac{n-m}{2m} \rfloor$. That is, cost 0 appears m times, cost p appears $2m$ times, cost $2p$ appears $2m$ times, and so on. Given an accepted job set $\mathcal{A}$, Lemma 6.1 enables us to optimally match the jobs in $\mathcal{A}$ to the first $|\mathcal{A}|$ costs in $\mathcal{C}$ by their weights. Note that for each job J_j, it can be either accepted or rejected. Next, we provide a DP algorithm (**DP$_3$**) to solve problem $Pm|rej, d_j = d, P_{\text{sum}} \le d, p_j = p|Z_{wr}$.

Algorithm DP$_3$: Number the n jobs in $\mathcal{J}$ according to the largest weight rule such that $w_1 \ge w_2 \ge \cdots \ge w_n$. Define $F(j, k)$ as the minimum integrated cost for a partial solution that includes the first j jobs and accepts exactly k of them.

The initial condition is $F(0, 0) = 0$.

The recursive function for $j = 1, 2, \ldots, n$ and $k = 0, 1, \ldots, j$ is given by

$$F(j,k) = \begin{cases} F(j-1,k) + r_j, & \text{if} \quad k = 0; \\ \min\{F(j-1,k) + r_j, F(j-1,k-1) + w_j c_k\}, & \text{if} \quad 1 \le k \le j-1; \\ F(j-1,k-1) + w_j c_k, & \text{if} \quad k = j. \end{cases}$$

The minimum solution value of (1) is given by $\min\{F(n, k) : k = 0, 1, \ldots, n\}$.

Theorem 6.1. *Algorithm* **DP$_3$** *solves problem* $Pm|rej, d_j = d, P_{\text{sum}} \le d, p_j = p|Z_{wr}$ *in* $O(n^2)$ *time.*

Proof. The correctness of **DP$_3$** follows from Lemmas 3.1, 3.2, 3.3 and 6.1, along with the preceding discussion. Since there are $O(n^2)$ different states (j, k) and each $F(j, k)$ is computable in constant time, the overall running time is $O(n^2)$. □

6.2 $Pm|rej, d_j = d, P_{\text{sum}} \le d, w_j = w|Z_{wr}$

We address the case where all jobs have equal weights. This case can be denoted by $Pm|rej, d_j = d, P_{\text{sum}} \le d, w_j = w|Z_{wr}$. To solve the problem, we first examine the schedule π_i on M_i more closely. Let $\pi_e^i = [J_{(1)}^i \prec J_{(2)}^i \prec \cdots \prec J_{(n_e^i)}^i]$ be the processing sequence of jobs in $\mathcal{A}_i^E$ on M_i, and let $\pi_t^i = [J_{[n_t^i]}^i \prec J_{[n_t^i-1]}^i \prec \cdots \prec J_{[1]}^i]$ be the processing sequence of jobs in $\mathcal{A}_i^T$ on M_i. See Figure 3 for an illustration of $\pi_i = (\pi_e^i, \pi_t^i)$ on M_i.

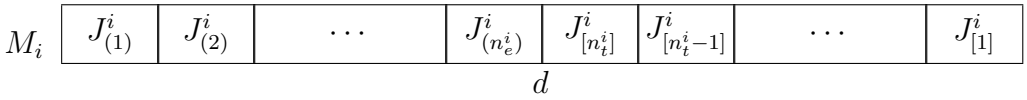

Figure 3: The schedule $\pi_i = (\pi_e, \pi_t)$ on M_i

Follows Hall (1986) and Shabtay et al. (2012), we introduce a *positional penalty* for each given scheduled position. Specifically, define $\chi_{(j)} = (j-1)w$ for any job scheduled in the j-th early position (from the left), and define $\chi_{[j]} = jw$ for any job scheduled in the j-th tardy position (from the right). Then the contribution of $J_{(j)}^i$ to the objective function (1) is $\chi_{(j)}p_{(j)}^i = (j-1)wp_{(j)}^i$, and the contribution of $J_{[j]}^i$ to (1) is $\chi_{[j]}p_{[j]}^i = jwp_{[j]}^i$. Hence, to minimize (1), we only need to select the n smallest positional penalties from the m machines. Let $\mathcal{X} = \{\chi_1, \chi_2, \ldots, \chi_n\}$ be the set of these n smallest costs, and assume without loss of generality that $\chi_1 \le \chi_2 \le \cdots \le \chi_n$. Then we have $\chi_1 = \chi_2 = \cdots = \chi_m = 0$, $\chi_{m+1} = \chi_{m+2} = \cdots = \chi_{3m} = w$, $\cdots$, $\chi_{n-v} = \chi_{n-v+1} = \cdots = \chi_n = (\lfloor \frac{n-m}{2m} \rfloor + 1)w$, where $v = n - m - 2m\lfloor \frac{n-m}{2m} \rfloor$. That is, penalty 0 appears m times, penalty w appears $2m$ times, penalty $2w$ appears $2m$ times, and so on. Given an accepted job set $\mathcal{A}$, Lemma 6.1 enables us to optimally match the jobs in $\mathcal{A}$ to the first $|\mathcal{A}|$ penalties in $\mathcal{X}$ by their processing times. Similar to $\mathbf{DP_3}$, the following DP algorithm ($\mathbf{DP_4}$) is provided to solve problem $Pm|rej, d_j = d, P_{\text{sum}} \le d, w_j = w|Z_{wr}$.

Algorithm $\mathbf{DP_4}$: Number the n jobs in $\mathcal{J}$ according to the largest processing time rule such that $p_1 \ge p_2 \ge \cdots \ge p_n$. Define $G(j,k)$ as the minimum integrated cost for a partial solution that includes the first j jobs and accepts exactly k of them.

The initial condition is $G(0,0) = 0$.

The recursive function for $j = 1, 2, \ldots, n$ and $k = 0, 1, \ldots, j$ is given by

$$G(j,k) = \begin{cases} G(j-1,k) + r_j, & \text{if} \quad k = 0; \\ \min\{G(j-1,k) + r_j, G(j-1,k-1) + w_j\chi_k\}, & \text{if} \quad 1 \le k \le j-1; \\ G(j-1,k-1) + w_j\chi_k, & \text{if} \quad k = j. \end{cases}$$

The minimum solution value of (1) is given by $\min\{G(n,k) : k = 0, 1, \ldots, n\}$.

Similar to the proof of Theorem 6.1, the following result holds.

Theorem 6.2. *Algorithm* $\mathbf{DP_4}$ *solves problem* $Pm|rej, d_j = d, P_{\text{sum}} \leq d, w_j = w|Z_{wr}$ *in* $O(n^2)$ *time.*

7 Conclusion

We study ET scheduling with optional job rejection on m identical parallel machines, where all jobs share an unrestricted CON. The goal is to minimize the integrated cost, which includes the TWET of accepted jobs and total penalty of rejected jobs. The key results of this paper are outlined as follows: (i) problem $Pm|d_j = d, P_{\text{sum}} \leq d, w_j = p_j|\sum w_j(E_j + T_j)$ is proved to be strongly $\mathcal{NP}$-hard for arbitrary m; (ii) problem $1|rej, d_j = d, P_{\text{sum}} \leq d, w_j = p_j|Z_{wr}$ is proved to be binary $\mathcal{NP}$-hard; (iii) when m is fixed, two pseudo-polynomial DP algorithms and two FPTASs are designed for problem $Pm|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$; (iv) $O(n^2)$-time algorithms are provided for two special cases.

As the time complexity of the exact DP algorithms and FPTASs designed for problem $Pm|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$ is high for larger values of m, it is challenging to design other exact or heuristic algorithms with reduced time complexity. Another interesting direction is to explore the problem of minimizing the TWET subject to a given upper bound on the total penalty of rejected jobs. Third, one can investigate a more general scenario where the weights of earliness and tardiness are asymmetric, i.e., $Pm|rej, d_j = d, P_{\text{sum}} \leq d|\sum_{J_j \in \mathcal{A}}(\alpha_j E_j + \beta_j T_j) + \sum_{J_j \in \mathcal{R}} r_j$.

Acknowledgments The authors thank the two anonymous reviewers for their valuable comments and suggestions, which have led to significant improvements in the quality and presentation of this paper.

Funding This work was supported by the National Natural Science Foundation of China (Nos. 12401427, 12471305), the National Natural Science Foundation of Henan Province (No. 262300420326), the Key Research Project of Henan Higher Education Institutions (No. 25B110010), and the Exploration and Practice of Application-Oriented Discrete Mathematics in Big Data (No. 2024ZGJGLX028).

Data availibility No data was used for the research described in the article.

Conflict of interest The authors declare that they have no conflict of interest.

Appendix A

To simplify the presentation before proving Lemmas 3.2-3.5, we introduce the following notation used for π^*. For each machine M_i $(i = 1, 2, \ldots, m)$, let J_f^i denote the first early job, J_e^i the job completing exactly at time d, J_g^i the first job finishing after time d, and J_l^i the last job. The schedule configuration of π^* on M_i is illustrated in Figure 4. Note that J_f^i and J_e^i may refer to the same job. Similarly, J_g^i and J_l^i may also refer to the same job.

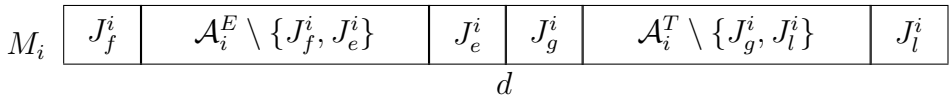

Figure 4: The schedule configuration of π^* on M_i

Proof of Lemma 3.2. We first claim that $P_{\text{sum}}(\mathcal{A}_i^T) \leq P_{\text{sum}}(\mathcal{A}_i^E)$ holds for each $i = 1, 2, \ldots, m$. Suppose to the contrary that there exists an index i_0 with $P_{\text{sum}}(\mathcal{A}_{i_0}^T) > P_{\text{sum}}(\mathcal{A}_{i_0}^E)$. We build a new solution π' from π^* by moving $J_l^{i_0}$ on M_{i_0} to the position immediately before $J_f^{i_0}$. Then, the change in the objective cost is $\Delta = Z_{wr}(\pi') - Z_{wr}(\pi^*) = w_l^{i_0}(P_{\text{sum}}(\mathcal{A}_{i_0}^E) - P_{\text{sum}}(\mathcal{A}_{i_0}^T)) < 0$, which contradicts the optimality of π^*. Hence, the claim holds.

By the averaging principle, we obtain $\min\limits_{i=1,2,\ldots,m}\{P_{\text{sum}}(\mathcal{A}_i^T)\} \leq \frac{1}{m}\sum_{i=1}^m P_{\text{sum}}(\mathcal{A}_i^T) \leq \frac{1}{2m}\sum_{i=1}^m (P_{\text{sum}}(\mathcal{A}_i^T) + P_{\text{sum}}(\mathcal{A}_i^E)) = \frac{1}{2m}P_{\text{sum}}(\mathcal{A})$, which proves (3).

Define $i_1 = \arg\min\limits_{i=1,2,\ldots,m}\{P_{\text{sum}}(\mathcal{A}_i^T)\}$ and $i_2 = \arg\max\limits_{i=1,2,\ldots,m}\{P_{\text{sum}}(\mathcal{A}_i^T)\}$. To prove (4), assume for contradiction that it fails, i.e., $\max\limits_{i=1,2,\ldots,m}\{P_{\text{sum}}(\mathcal{A}_i^T)\} = P_{\text{sum}}(\mathcal{A}_{i_2}^T) > \frac{1}{2m}P_{\text{sum}}(\mathcal{A}) + p_{\max}(\mathcal{A})$. We build a new solution π'' from π^* by moving $J_l^{i_2}$ from M_{i_2} to the position immediately after $J_l^{i_1}$ on M_{i_1}. Then, the change in the objective cost is $\Delta = Z_{wr}(\pi'') - Z_{wr}(\pi^*) = w_l^{i_2}(P_{\text{sum}}(\mathcal{A}_{i_1}^T) + p_l^{i_2} - P_{\text{sum}}(\mathcal{A}_{i_2}^T))$. Since $p_l^{i_2} \leq p_{\max}(\mathcal{A})$ and $P_{\text{sum}}(\mathcal{A}_{i_1}^T) \leq \frac{1}{2m}P_{\text{sum}}(\mathcal{A})$, we obtain $\Delta \leq w_l^{i_2}(\frac{1}{2m}P_{\text{sum}}(\mathcal{A}) + p_{\max}(\mathcal{A}) - P_{\text{sum}}(\mathcal{A}_{i_2}^T)) < 0$, contradicting the optimality of π^*. Hence, (4) holds. □

Proof of Lemma 3.3. We first show that

$$P_{\text{sum}}(\mathcal{A}_i^E) \leq P_{\text{sum}}(\mathcal{A}_k^T) + 2p_f^i, \ i, k \in \{1, 2, \ldots, m\}. \tag{25}$$

Suppose to the contrary that there exist two indices i_0 and k_0 (possibly equal) such that $P_{\text{sum}}(\mathcal{A}_{i_0}^E) > P_{\text{sum}}(\mathcal{A}_{k_0}^T) + 2p_f^{i_0}$. We build a new solution π' from π^* by moving $J_f^{i_0}$ on M_{i_0} to the position immediately after $J_l^{k_0}$. Then, the change in the objective cost is $\Delta = Z_{wr}(\pi') - Z_{wr}(\pi^*) = w_f^{i_0}(P_{\text{sum}}(\mathcal{A}_{k_0}^T) + p_f^{i_0}) - (P_{\text{sum}}(\mathcal{A}_{i_0}^T) - p_f^{i_0})) < 0$, which contradicts the optimality of π^*. Hence, (25) holds.

In particular, taking $k = i$ in (25) gives $P_{\text{sum}}(\mathcal{A}_i^E) \leq P_{\text{sum}}(\mathcal{A}_i^T) + 2p_f^i \leq P_{\text{sum}}(\mathcal{A}_i^T) + 2p_{\max}(\mathcal{A})$ for each $i = 1, 2, \ldots, m$. By the averaging principle, we obtain

$\min_{i=1,2,\ldots,m}\{P_{\rm sum}(\mathcal{A}_i^E)\} \le \frac{1}{m}\sum_{i=1}^m P_{\rm sum}(\mathcal{A}_i^E) \le \frac{1}{2m}\sum_{i=1}^m (P_{\rm sum}(\mathcal{A}_i^E) + P_{\rm sum}(\mathcal{A}_i^T) + 2p_{\max}(\mathcal{A})) = \frac{1}{2m}P_{\rm sum}(\mathcal{A}) + p_{\max}(\mathcal{A})$, which proves (5).

Define $i_3 = \arg\max_{i=1,2,\ldots,m}\{P_{\rm sum}(\mathcal{A}_i^E)\}$. In view of (3) and $p_f^{i_3} \le p_{\max}(\mathcal{A})$, we apply (25) with $k = i_3$ and $i = i_1$ to obtain $\max_{i=1,2,\ldots,m}\{P_{\rm sum}(\mathcal{A}_i^E)\} = P_{\rm sum}(\mathcal{A}_{i_3}^E) \le P_{\rm sum}(\mathcal{A}_{i_1}^T) + 2p_f^{i_3} \le \frac{1}{2m}P_{\rm sum}(\mathcal{A}) + 2p_{\max}(\mathcal{A})$, which proves (6). □

Proof of Lemma 3.4. We first claim that $W_{\rm sum}(\mathcal{A}_i^T) \le W_{\rm sum}(\mathcal{A}_i^E)$ holds for each $i = 1, 2, \ldots, m$. Suppose to the contrary that there exists an index i_0 with $W_{\rm sum}(\mathcal{A}_{i_0}^T) > W_{\rm sum}(\mathcal{A}_{i_0}^E)$. We build a new solution π' from π^* by left-shifting all jobs on M_{i_0} by δ_1 time units, where $0 < \delta_1 < p_g^{i_0}$. Then, the change in the objective cost is $\Delta = Z_{wr}(\pi') - Z_{wr}(\pi^*) = \delta_1(W_{\rm sum}(\mathcal{A}_{i_0}^E) - W_{\rm sum}(\mathcal{A}_{i_0}^T)) < 0$, contradicting the optimality of π^*. Hence, the claim holds.

By the averaging principle, we obtain $\min_{i=1,2,\ldots,m}\{W_{\rm sum}(\mathcal{A}_i^T)\} \le \frac{1}{m}\sum_{i=1}^m W_{\rm sum}(\mathcal{A}_i^T) \le \frac{1}{2m}\sum_{i=1}^m (W_{\rm sum}(\mathcal{A}_i^T) + W_{\rm sum}(\mathcal{A}_i^E)) = \frac{1}{2m}W_{\rm sum}(\mathcal{A})$, which proves (7).

Define $k_1 = \arg\min_{i=1,2,\ldots,m}\{W_{\rm sum}(\mathcal{A}_i^T)\}$ and $k_2 = \arg\max_{i=1,2,\ldots,m}\{W_{\rm sum}(\mathcal{A}_i^T)\}$. To prove (8), assume for contradiction that it fails, i.e., $\max_{i=1,2,\ldots,m}\{W_{\rm sum}(\mathcal{A}_i^T)\} = W_{\rm sum}(\mathcal{A}_{k_2}^T) > \frac{1}{2m}W_{\rm sum}(\mathcal{A}) + w_{\max}(\mathcal{A})$. We build a new solution π'' from π^* by moving $J_g^{k_2}$ from M_{k_2} to the position immediately before $J_g^{k_1}$ on M_{k_1}, ensuring $J_g^{k_2}$ starts at time d. Then, the change in the objective cost is $\Delta = Z_{wr}(\pi'') - Z_{wr}(\pi^*) = p_g^{k_2}(W_{\rm sum}(\mathcal{A}_{k_1}^T) - (W_{\rm sum}(\mathcal{A}_{k_2}^T) - w_g^{k_2}))$. Since $w_g^{k_2} \le w_{\max}(\mathcal{A})$ and $W_{\rm sum}(\mathcal{A}_{k_1}^T) \le \frac{1}{2m}W_{\rm sum}(\mathcal{A})$, we obtain $\Delta \le p_g^{k_2}(\frac{1}{2m}W_{\rm sum}(\mathcal{A}) + w_{\max}(\mathcal{A}) - W_{\rm sum}(\mathcal{A}_{k_2}^T)) < 0$, contradicting the optimality of π^*. Hence, (8) holds. □

Proof of Lemma 3.5. We first claim that $W_{\rm sum}(\mathcal{A}_i^E) \le W_{\rm sum}(\mathcal{A}_i^T) + 2w_g^i$ holds for each $i = 1, 2, \ldots, m$. Suppose to the contrary that there exists an index i_0 with $W_{\rm sum}(\mathcal{A}_{i_0}^E) > W_{\rm sum}(\mathcal{A}_{i_0}^T) + 2w_g^{i_0}$. We build a new solution π' from π^* by delaying all jobs on M_{i_0} by δ_2 time units, where $0 < \delta_2 < p_e^{i_0}$. Then, the change in the objective cost is $\Delta = Z_{wr}(\pi') - Z_{wr}(\pi^*) = \delta_2((W_{\rm sum}(\mathcal{A}_{i_0}^T) + w_g^{i_0}) - (W_{\rm sum}(\mathcal{A}_{i_0}^E) - w_g^{i_0})) < 0$, which contradicts the optimality of π^*. Hence, the claim holds.

By the averaging principle, we obtain $\min_{i=1,2,\ldots,m}\{W_{\rm sum}(\mathcal{A}_i^E)\} \le \frac{1}{m}\sum_{i=1}^m W_{\rm sum}(\mathcal{A}_i^E) \le \frac{1}{2m}\sum_{i=1}^m (W_{\rm sum}(\mathcal{A}_i^E) + W_{\rm sum}(\mathcal{A}_i^T) + 2w_g^i) \le \frac{1}{2m}W_{\rm sum}(\mathcal{A}) + w_{\max}(\mathcal{A})$, which proves (9).

Define $k_3 = \arg\min_{i=1,2,\ldots,m}\{W_{\rm sum}(\mathcal{A}_i^E)\}$ and $k_4 = \arg\max_{i=1,2,\ldots,m}\{W_{\rm sum}(\mathcal{A}_i^E)\}$. To prove (10), assume for contradiction that it fails, i.e., $\max_{i=1,2,\ldots,m}\{W_{\rm sum}(\mathcal{A}_i^E)\} = W_{\rm sum}(\mathcal{A}_{k_4}^E) > \frac{1}{2m}W_{\rm sum}(\mathcal{A}) + 2w_{\max}(\mathcal{A})$. We build a new solution π'' from π^* by moving $J_e^{k_4}$ from M_{k_4} to the position just after $J_e^{k_3}$ on M_{k_3}, ensuring $J_e^{k_4}$ completes at time d. Then, the change in the objective cost is $\Delta = Z_{wr}(\pi'') - Z_{wr}(\pi^*) = p_e^{k_4}(W_{\rm sum}(\mathcal{A}_{k_3}^E) - (W_{\rm sum}(\mathcal{A}_{k_4}^E) - w_e^{k_4}))$. Since $w_e^{k_4} \le w_{\max}(\mathcal{A})$

and $W_{\text{sum}}(\mathcal{A}^E_{k_3}) \leq \frac{1}{2m}W_{\text{sum}}(\mathcal{A}) + w_{\max}(\mathcal{A})$, we obtain $\Delta \leq p_e^{k_4}((\frac{1}{2m}W_{\text{sum}}(\mathcal{A}) + w_{\max}(\mathcal{A})) - (W_{\text{sum}}(\mathcal{A}^E_{k_4}) - w_{\max}(\mathcal{A}))) < 0$, contradicting the optimality of π^*. Hence, (10) holds. □

Appendix B

$$
\begin{aligned}
Z_{wr} &= \sum_{j=1}^{n} w_j (\sum_{i=1}^{m} x_{ij} \sum_{k=1}^{j-1} p_k x_{ik} + \sum_{i=1}^{m} y_{ij} \sum_{k=1}^{j} p_k y_{ik}) + \sum_{j=1}^{n} r_j z_j \\
&= \sum_{i=1}^{m} (\sum_{j=1}^{n} w_j x_{ij} \sum_{k=1}^{j-1} p_k x_{ik} + \sum_{j=1}^{n} w_j y_{ij} \sum_{k=1}^{j} p_k y_{ik}) + \sum_{j=1}^{n} r_j z_j \\
&= \sum_{i=1}^{m} (\sum_{k=1}^{n} p_k x_{ik} \sum_{j=k+1}^{n} w_j x_{ij} + \sum_{k=1}^{n} p_k y_{ik} \sum_{j=k}^{n} w_j y_{ij}) + \sum_{j=1}^{n} r_j z_j \\
&= \sum_{i=1}^{m} (\sum_{j=1}^{n} p_j x_{ij} \sum_{k=j+1}^{n} w_k x_{ik} + \sum_{j=1}^{n} p_j y_{ij} \sum_{k=j}^{n} w_k y_{ik}) + \sum_{j=1}^{n} r_j z_j \\
&= \sum_{j=1}^{n} p_j (\sum_{i=1}^{m} x_{ij} \sum_{k=j+1}^{n} w_k x_{ik} + \sum_{i=1}^{m} y_{ij} \sum_{k=j}^{n} w_k y_{ik}) + \sum_{j=1}^{n} r_j z_j.
\end{aligned}
$$

Appendix C

Below, we illustrate the detailed implementation of $\mathbf{DP_1}$ on the instance of $1|rej, d_j = d, P_{\text{sum}} \leq d|Z_{wr}$ with three jobs, where $(p_1, w_1, r_1) = (6, 9, 38)$, $(p_2, w_2, r_2) = (7, 7, 40)$, and $(p_3, w_3, r_3) = (7, 5, 50)$.

[Preprocessing] The three jobs are already numbered such that $\frac{p_1}{w_1} \leq \frac{p_2}{w_2} \leq \frac{p_3}{w_3}$.

[Initialization] Set $\Phi_0 = \{(0, 0, 0)\}$ and $\Phi_j = \emptyset$ for $j = 1, 2, 3$.

[Recursion] Generate sets Φ_1 to Φ_3.

For $j = 1$: from $(0, 0, 0) \in \Phi_0$, the check in line 16 fails since $9 \times 6 = 54 > 38$, thus only add $(6, 0, 0)$ and $(0, 0, 38)$ to Φ_1. Consequently, $\Phi_1 = \{(0, 0, 38), (6, 0, 0)\}$.

For $j = 2$: from $(0, 0, 38) \in \Phi_1$, the check in line 16 fails since $7 \times 7 = 49 > 40$, thus only add $(7, 0, 38)$ and $(0, 0, 78)$ to Φ_2; from $(6, 0, 0) \in \Phi_1$, the check in line 11 fails since $7 \times 6 = 42 > 40$ and the check in line 16 fails since $7 \times 7 = 49 > 40$, thus only add $(6, 0, 40)$ to Φ_2. Consequently, $\Phi_2 = \{(0, 0, 78), (6, 0, 40), (7, 0, 38)\}$.

For $j = 3$: from $(0, 0, 78) \in \Phi_2$, add $(7, 0, 78)$, $(0, 7, 113)$, and $(0, 0, 128)$ to Φ_3; from $(6, 0, 40) \in \Phi_2$, add $(13, 0, 70)$, $(6, 7, 75)$, and $(6, 0, 90)$ to Φ_3; from $(7, 0, 38) \in \Phi_2$, add $(14, 0, 73)$, $(7, 7, 73)$, and $(7, 0, 88)$ to Φ_3. Hence, before elimination, $\Phi_3 = \{(0, 0, 128), (0, 7, 113), (6, 0, 90), (6, 7, 75), (7, 0, 78), (7, 0, 88), (7, 7, 73), (13, 0, 70), (14, 0, 73)\}$.

Since $(7, 0, 88)$ is dominated by $(7, 0, 78)$, after elimination (line 23), $\Phi_3 = \{(0, 0, 128), (0, 7, 113), (6, 0, 90), (6, 7, 75), (7, 0, 78), (7, 7, 73), (13, 0, 70), (14, 0, 73)\}$.

[Result] The optimal solution value is $z^* = 70$, corresponding to the tuple $(13, 0, 70) \in \Phi_3$.

References

Arik, O. A. (2023). A heuristic for single machine common due date assignment problem with different earliness/tardiness weights. *Opsearch*, 60(3):1561–1574.

Arik, O. A., Schutten, M., and Topan, E. (2022). Weighted earliness/tardiness parallel machine scheduling problem with a common due date. *Expert Systems with Applications*, 187:115916.

Atsmony, M. and Mosheiov, G. (2024a). Common due-date assignment problems with fixed-plus-linear earliness and tardiness costs. *Computers & Industrial Engineering*, 188:109915.

Atsmony, M. and Mosheiov, G. (2024b). Single machine scheduling to minimize maximum earliness/tardiness cost with job rejection. *Optimization Letters*, 18(3):751–766.

Cesaret, B., Oğuz, C., and Salman, F. S. (2012). A tabu search algorithm for order acceptance and scheduling. *Computers & Operations Research*, 39(6):1197–1205.

Chen, R. X. and Li, S. S. (2024). Two-machine job shop scheduling with optional job rejection. *Optimization Letters*, 18(7):1593–1618.

Fomin, A. and Goldengorin, B. (2022). An exact algorithm for the preemptive single machine scheduling of equal-length jobs. *Computers & Operations Research*, 142:105742.

Fowler, J. W. and Monch, L. (2022). A survey of scheduling with parallel batch (p-batch) processing. *European Journal of Operational Research*, 298(1):1–24.

Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of $\mathcal{NP}$-Completeness*. W. H. Freeman and Company, San Francisco.

Geng, Z. and Lu, L. (2025). Multiple identical serial-batch machines scheduling with release dates and submodular rejection penalties. *Journal of Combinatorial Optimization*, 49:32.

Hall, N. G. (1986). Single and multiple-processor models for minimizing completion time variance. *Naval Research Logistics*, 33(1):49–54.

Hall, N. G., Kubiak, W., and Sethi, S. P. (1991). Earliness–tardiness scheduling problems, II: Deviation of completion times about a restrictive common due date. *Operations Research*, 39(5):847–856.

Hall, N. G. and Posner, M. E. (1991). Earliness-tardiness scheduling problems, I: Weighted deviation of completion times about a common due date. *Operations Research*, 39(5):836–846.

Hardy, G. H., Littlewood, J. E., and Polya, G. (1967). *Inequalities*. Cambridge University Press, London.

Hoogeveen, J. A. and van de Velde, S. (1991). Scheduling around a small common due date. *European Journal of Operational Research*, 55(2):237–242.

Hou, B., Guo, T., Gao, S., Wang, G., Wu, W., and Liu, W. (2024). W-prize-collecting scheduling problem on parallel machines. *Journal of Combinatorial Optimization*, 48:26.

Hu, G., Pan, P., Liu, S., Yang, P., and Xie, R. (2024). The prize-collecting single machine scheduling with bounds and penalties. *Journal of Combinatorial Optimization*, 48:12.

Kacem, I. and Kellerer, H. (2024). Minimizing the maximum lateness for scheduling with release times and job rejection. *Journal of Combinatorial Optimization*, 48:23.

Kellerer, H., Rustogi, K., and Strusevich, V. A. (2020). A fast FPTAS for single machine scheduling problem of minimizing total weighted earliness and tardiness about a large common due date. *Omega*, 90:101992.

Kim, E. S. and Lee, I. S. (2022). Scheduling of two-machine flowshop with outsourcing lead-time. *Computers & Operations Research*, 145:105864.

Kramer, A. and Subramanian, A. (2019). A unified heuristic and an annotated bibliography for a large class of earliness–tardiness scheduling problems. *Journal of Scheduling*, 22(1):21–57.

Li, C. L. and Cheng, T. C. E. (1994). The parallel machine min-max weighted absolute lateness scheduling problem. *Naval Research Logistics*, 41(1):33–46.

Li, S. S. and Chen, R. X. (2020). Scheduling with common due date assignment to minimize generalized weighted earliness–tardiness penalties. *Optimization Letters*, 14(7):1681–1699.

Mor, B. and Shapira, D. (2022). Minsum scheduling with acceptable lead-times and optional job rejection. *Optimization Letters*, 16(3):1073–1091.

Mosheiov, G. and Sarig, A. (2024). A common due-date assignment problem with job rejection on parallel uniform machines. *International Journal of Production Research*, 62(6):2083–2092.

Nasrollahi, V., Moslehi, G., and Reisi-Nafchi, M. (2022). Minimizing the weighted sum of maximum earliness and maximum tardiness in a single-agent and two-agent form of a two-machine flow shop scheduling problem. *Operational Research*, 22:1403–1442.

Oğuz, C., Salman, F. S., and Yalcin, Z. B. (2010). Order acceptance and scheduling decisions in make-to-order systems. *International Journal of Production Economics*, 125(1):200–211.

Rolim, G. A. and Nagano, M. S. (2020). Structural properties and algorithms for earliness and tardiness scheduling against common due dates and windows: A review. *Computers & Industrial Engineering*, 149:106803.

Safarzadeh, H. and Kianfar, F. (2019). Job shop scheduling with the option of jobs outsourcing. *International Journal of Production Research*, 57(10):3255–3272.

Shabtay, D., Gaspar, N., and Kaspi, M. (2013). A survey on offline scheduling with rejection. *Journal of Scheduling*, 16(1):3–28.

Shabtay, D., Gaspar, N., and Yedidsion, L. (2012). A bicriteria approach to scheduling a single machine with job rejection and positional penalties. *Journal of Combinatorial Optimization*, 23(4):395–424.

Shabtay, D. and Gerstl, E. (2024). Coordinating scheduling and rejection decisions in a two-machine flow shop scheduling problem. *European Journal of Operational Research*, 316(3):887–898.

Slotnick, S. A. (2011). Order acceptance and scheduling: A taxonomy and review. *European Journal of Operational Research*, 222(1):1–11.

Sun, H. and Wang, G. (2003). Parallel machine earliness and tardiness scheduling with proportional weights. *Computers & Operations Research*, 30(5):801–808.